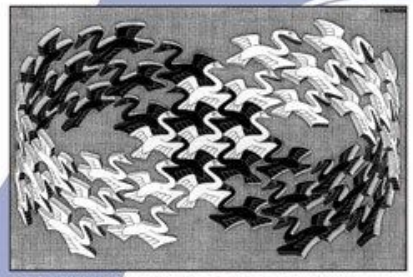
Design Patterns

Overview

Design Patterns

Elements of Reusable Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**

Structural

- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight

Behavioral

- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**

Structural

- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight

Behavioral

- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State



The Problem of CREATION

Context

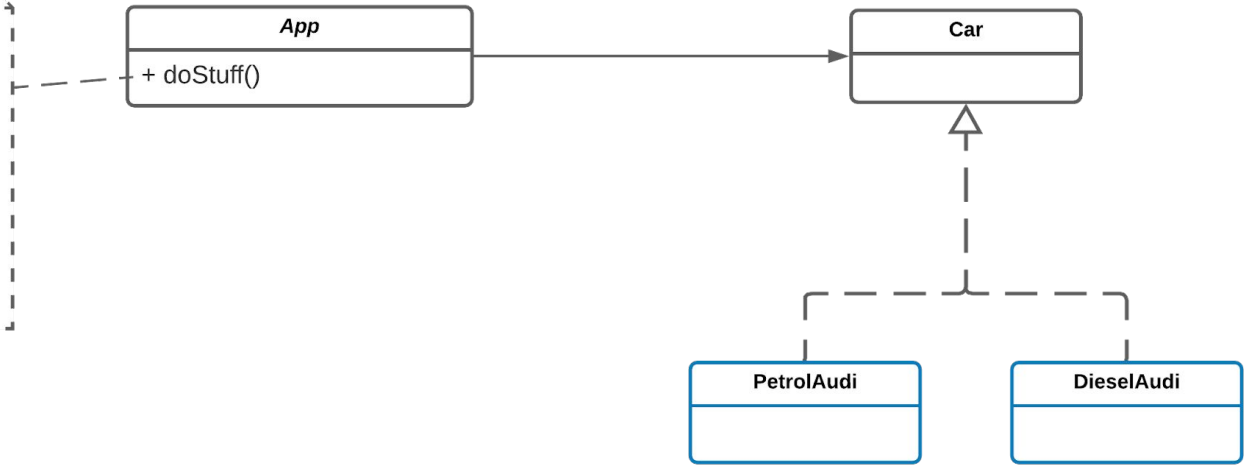
```
... // do something

Car petrolCar = new PetrolAudi();

... // do something

Car dieselCar = new DieselAudi();

... // do something
```



Context (DIP, OCP)

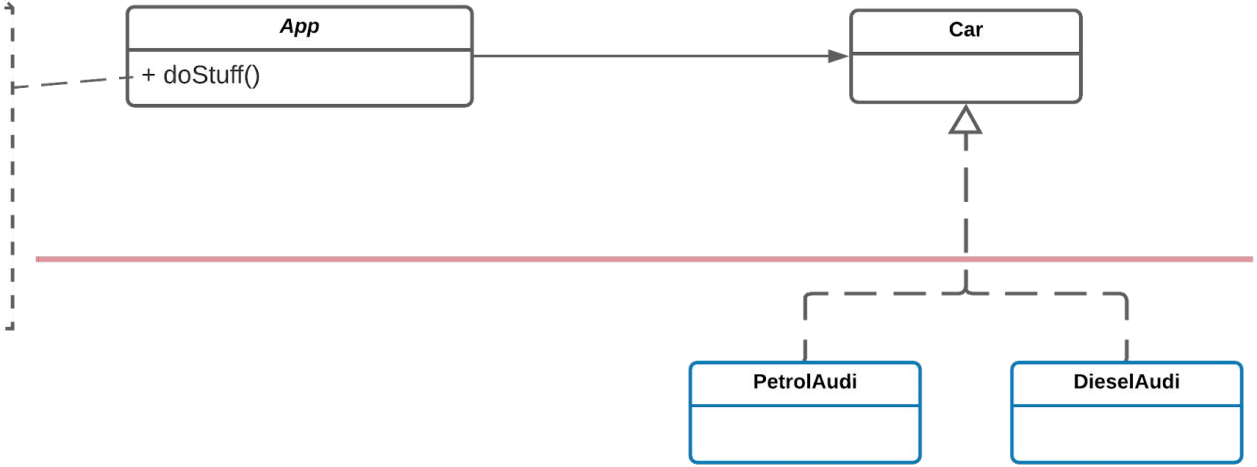
```
... // do something

Car petrolCar = new PetrolAudi();

... // do something

Car dieselCar = new DieselAudi();

... // do something
```



Problem?



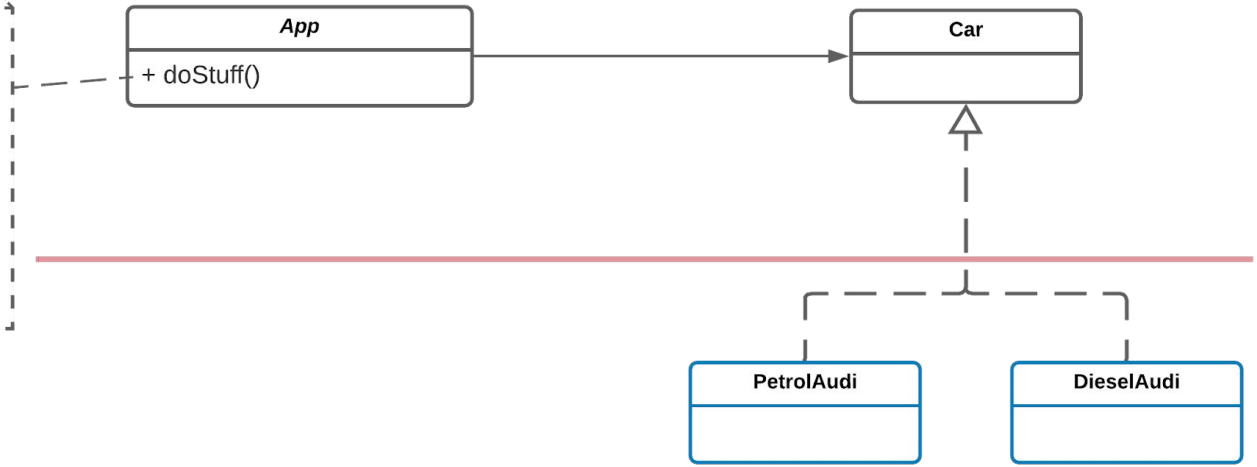
```
... // do something

Car petrolCar = new PetrolAudi();

... // do something

Car dieselCar = new DieselAudi();

... // do something
```



The CREATION Problem



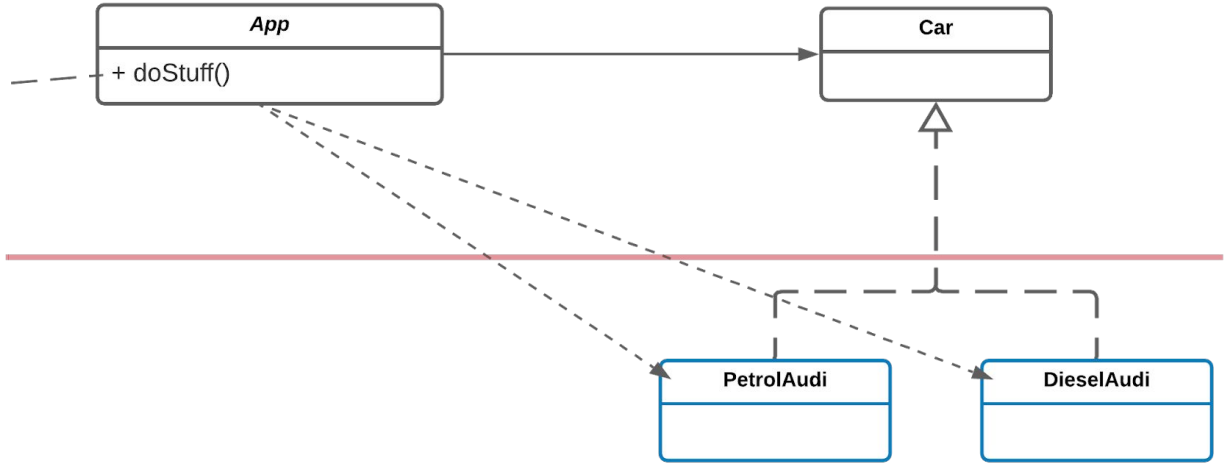
```
... // do something

Car petrolCar = new PetrolAudi();

... // do something

Car dieselCar = new DieselAudi();

... // do something
```



The CREATION Solution



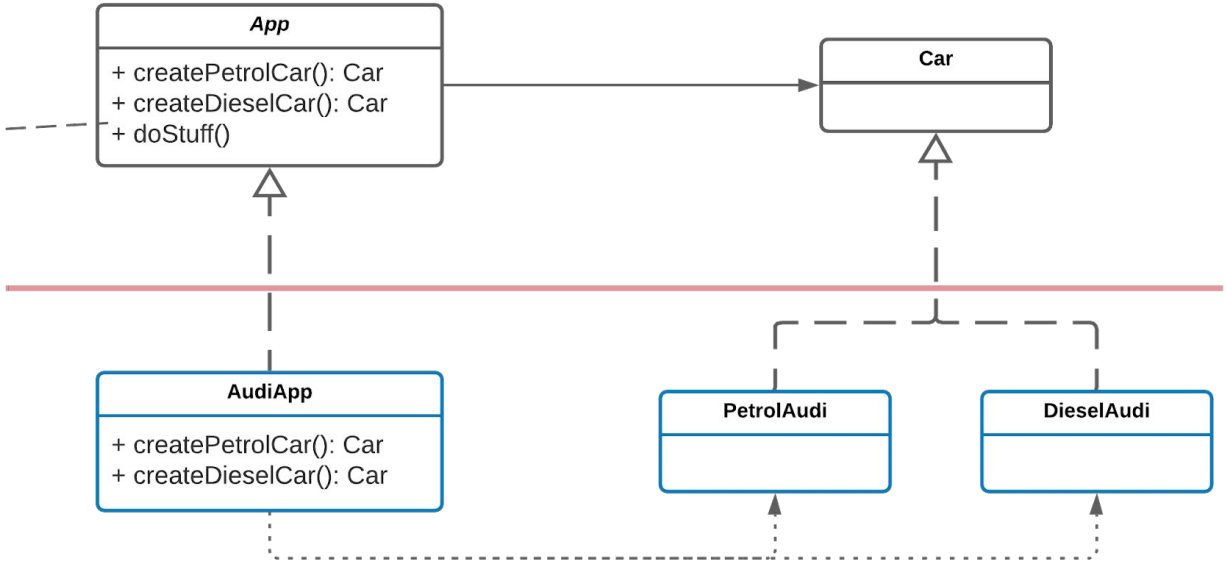
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Problem?



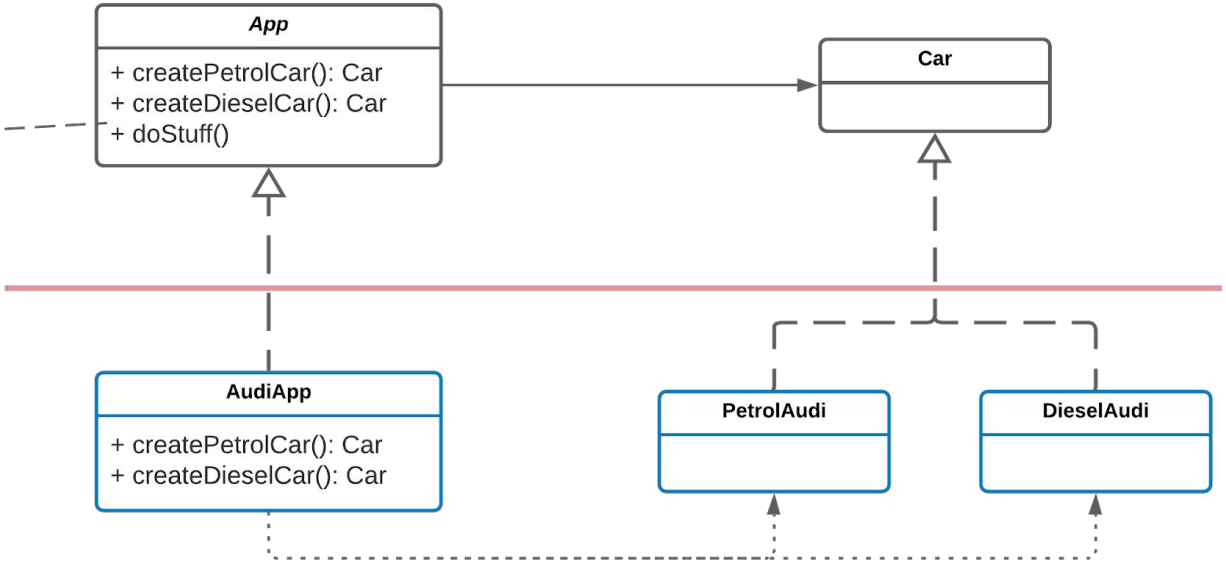
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Problem? Adapting for new Vendors...



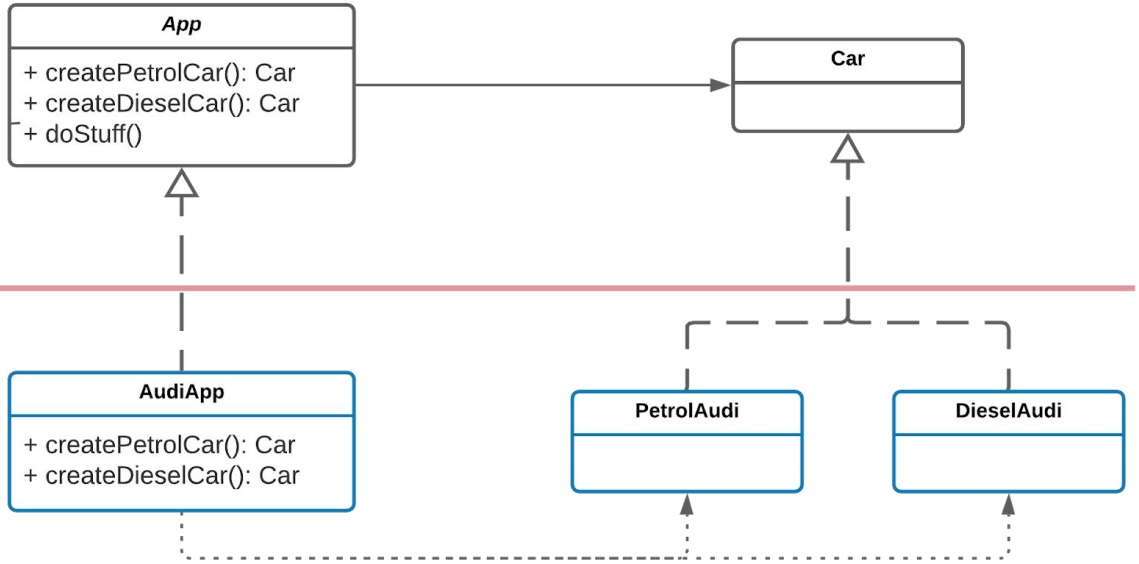
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Support Different Vendors



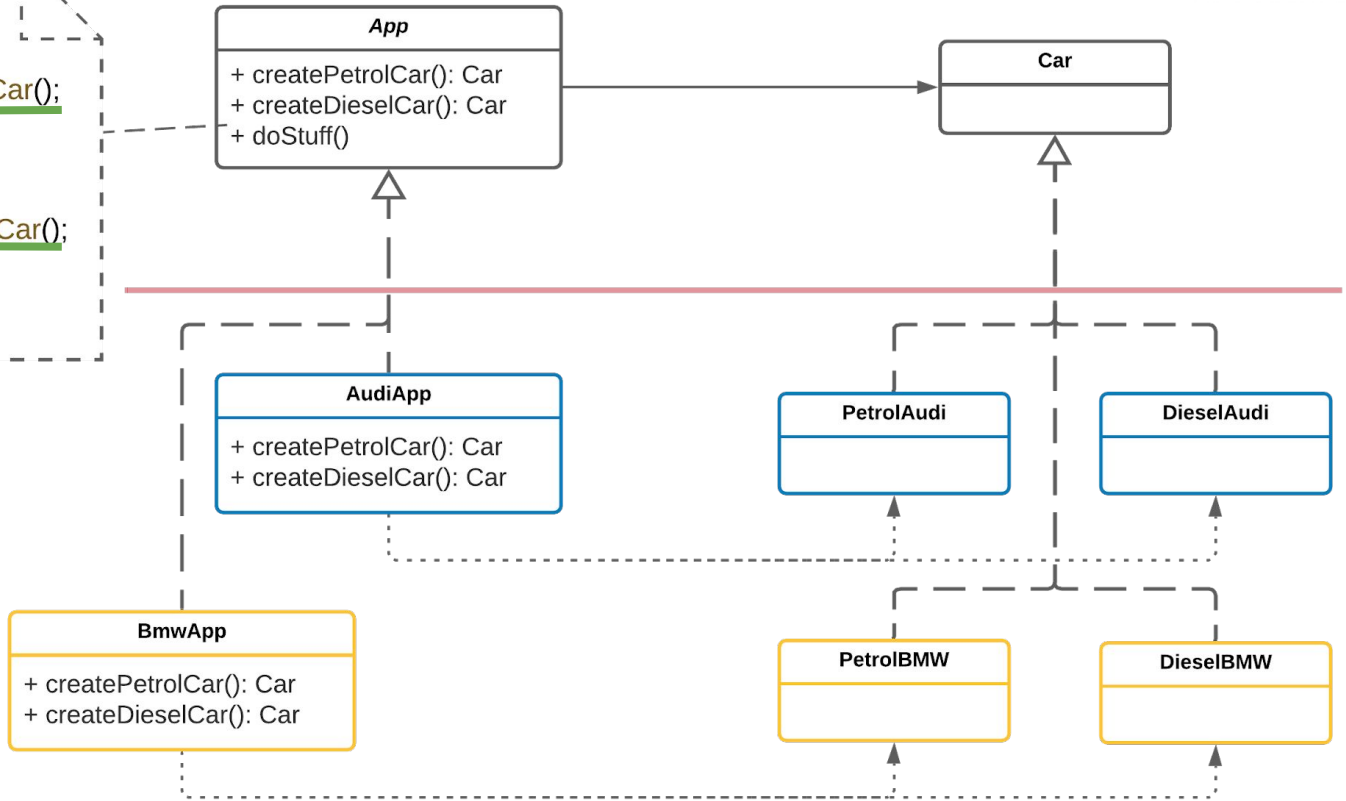
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Factory Method Pattern!



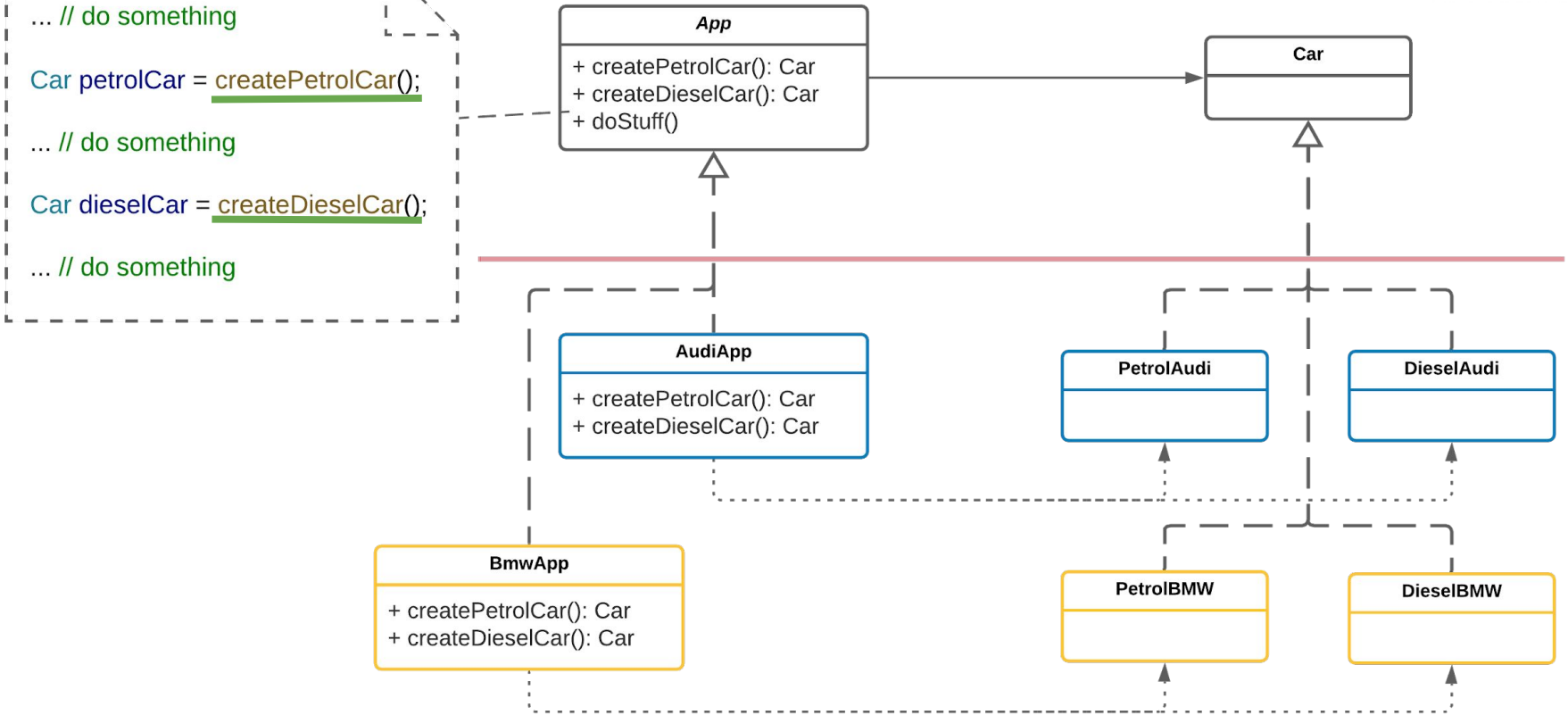
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Problem?...



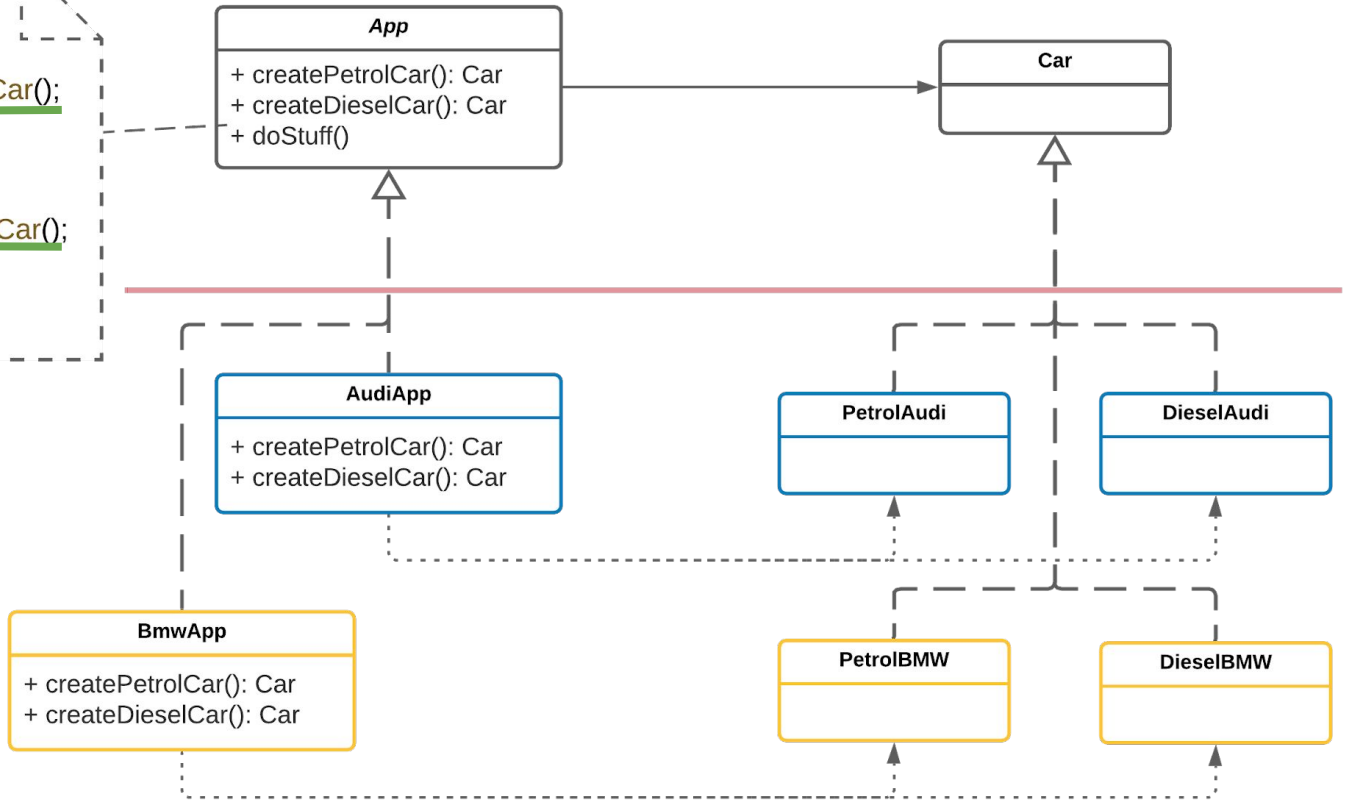
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```



Problem? Support BOTH Vendors?...



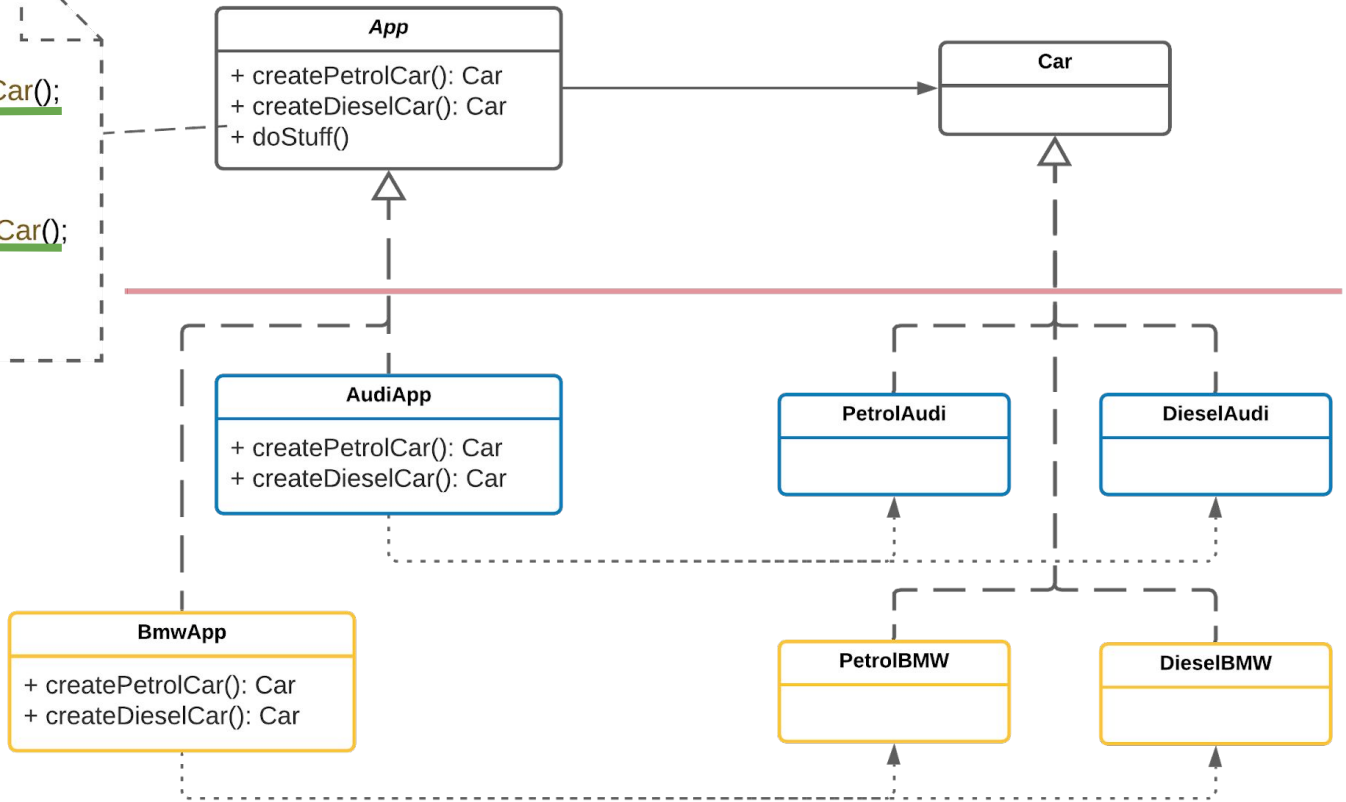
```
... // do something

Car petrolCar = createPetrolCar();

... // do something

Car dieselCar = createDieselCar();

... // do something
```





Favor composition over inheritance!

Favor Composition Over Inheritance...



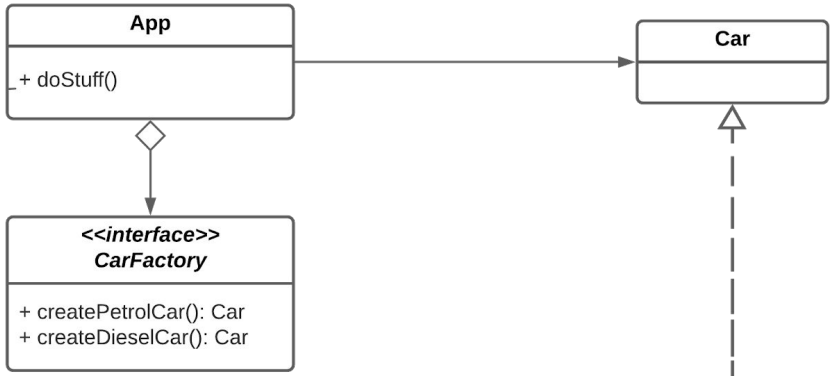
```
... // do something

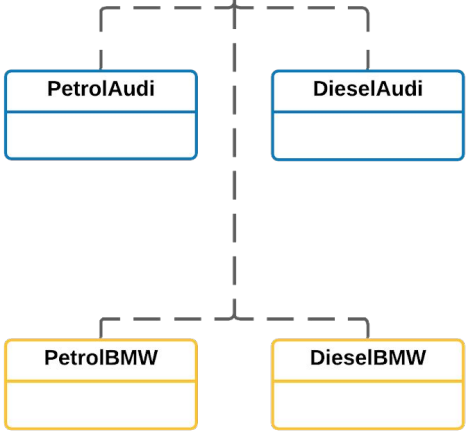
Car petrolCar = factory.createPetrolCar();

... // do something

Car dieselCar = factory.createDieselCar();

... // do something
```

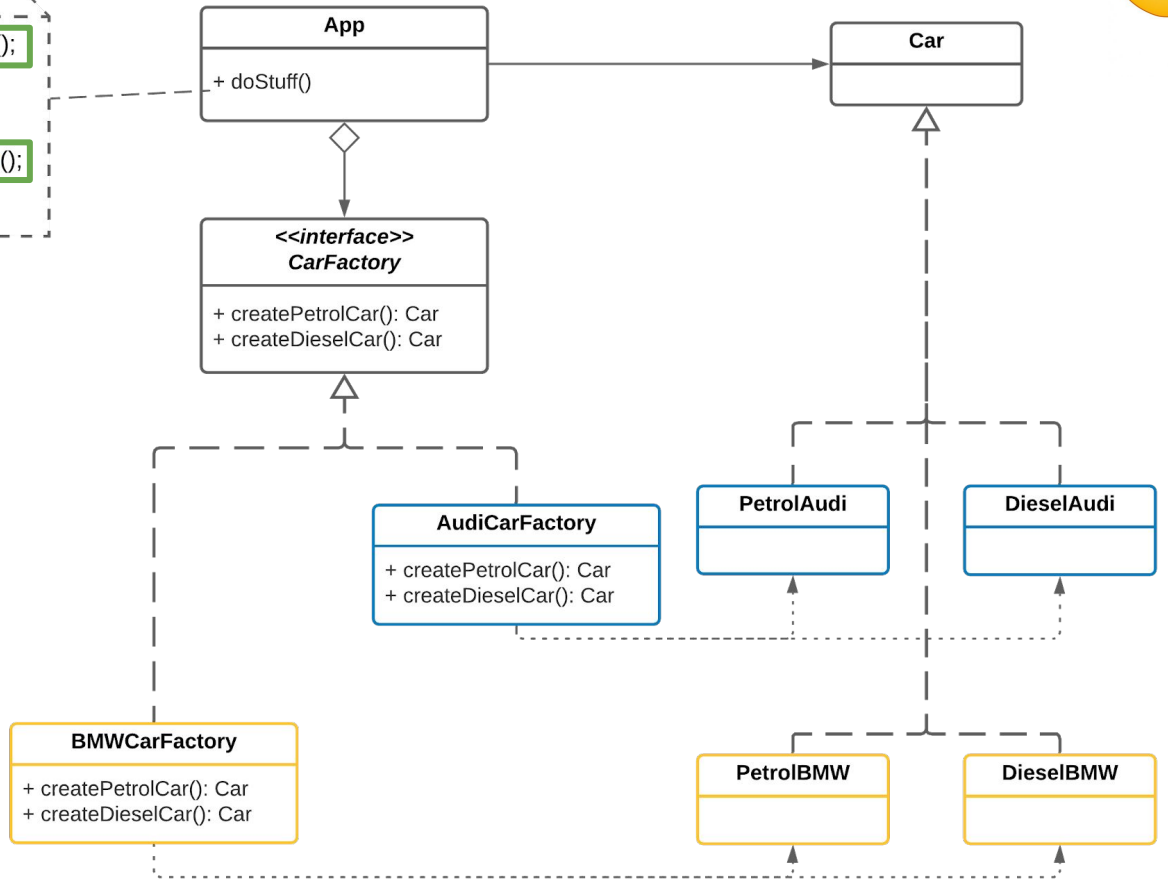




Favor Composition Over Inheritance...



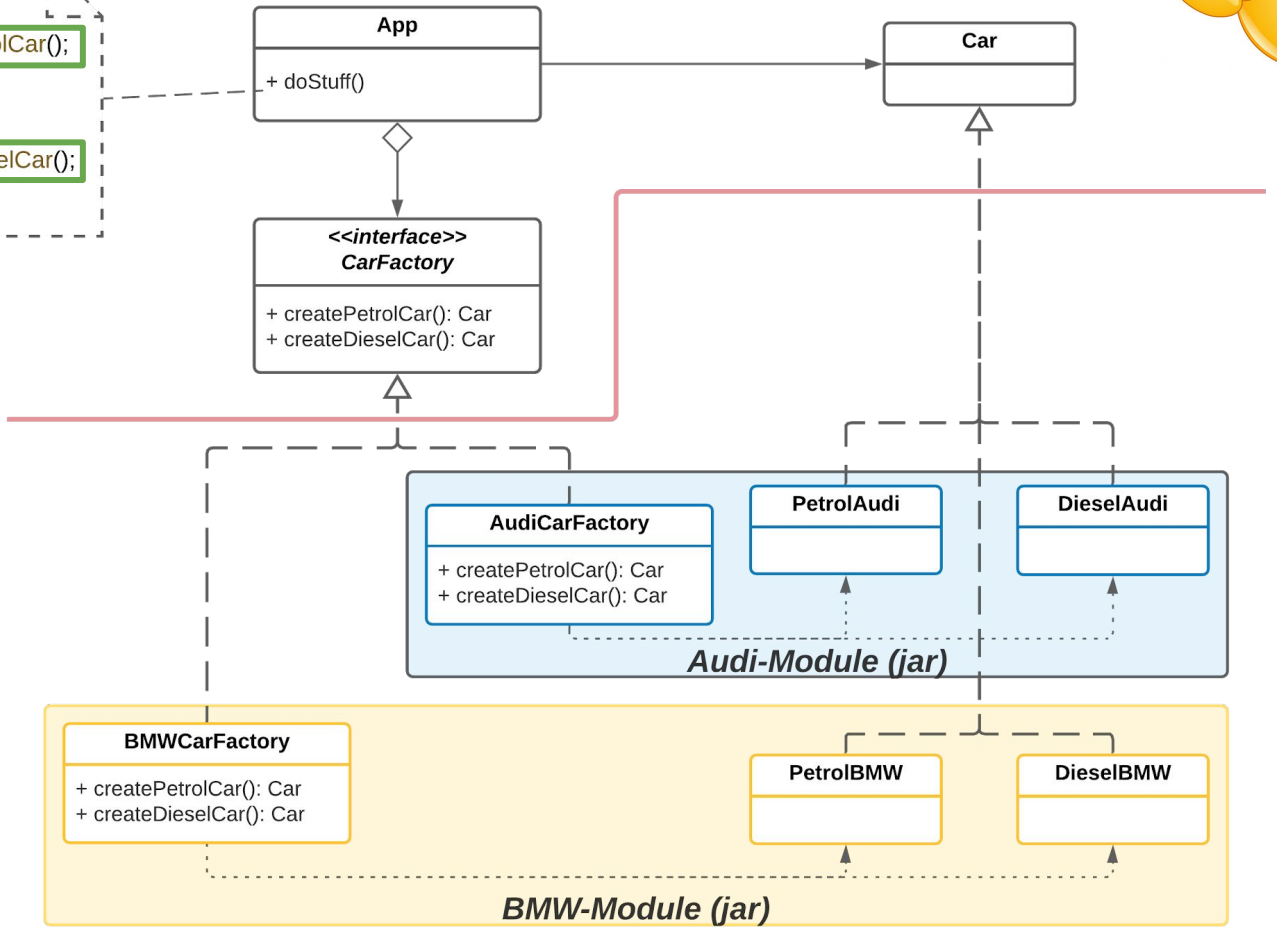
```
... // do something
Car petrolCar = factory.createPetrolCar();
... // do something
Car dieselCar = factory.createDieselCar();
... // do something
```



Independent deployability...



```
... // do something
Car petrolCar = factory.createPetrolCar();
... // do something
Car dieselCar = factory.createDieselCar();
... // do something
```



Abstract Factory Pattern!



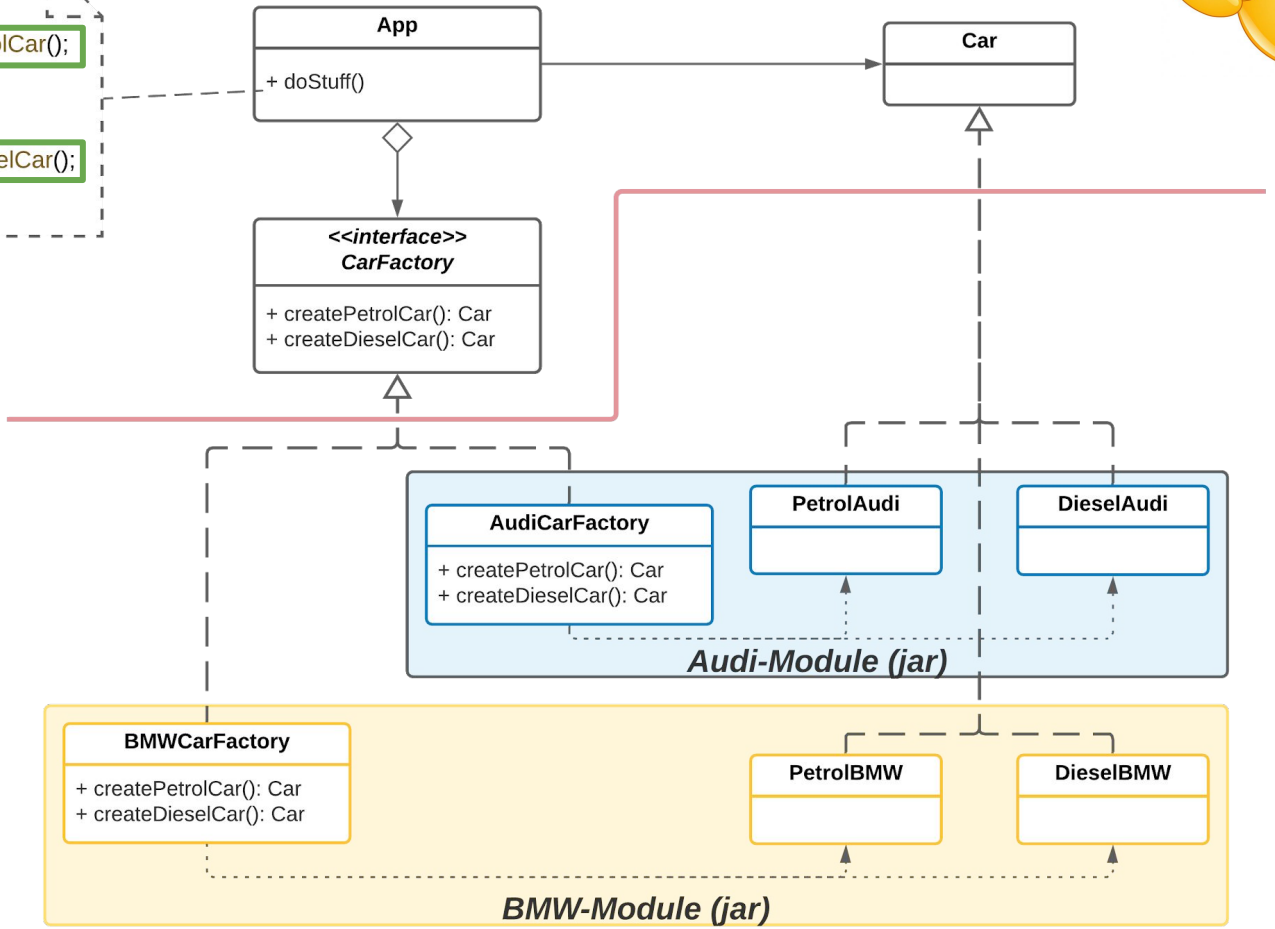
```
... // do something

Car petrolCar = factory.createPetrolCar();

... // do something

Car dieselCar = factory.createDieselCar();

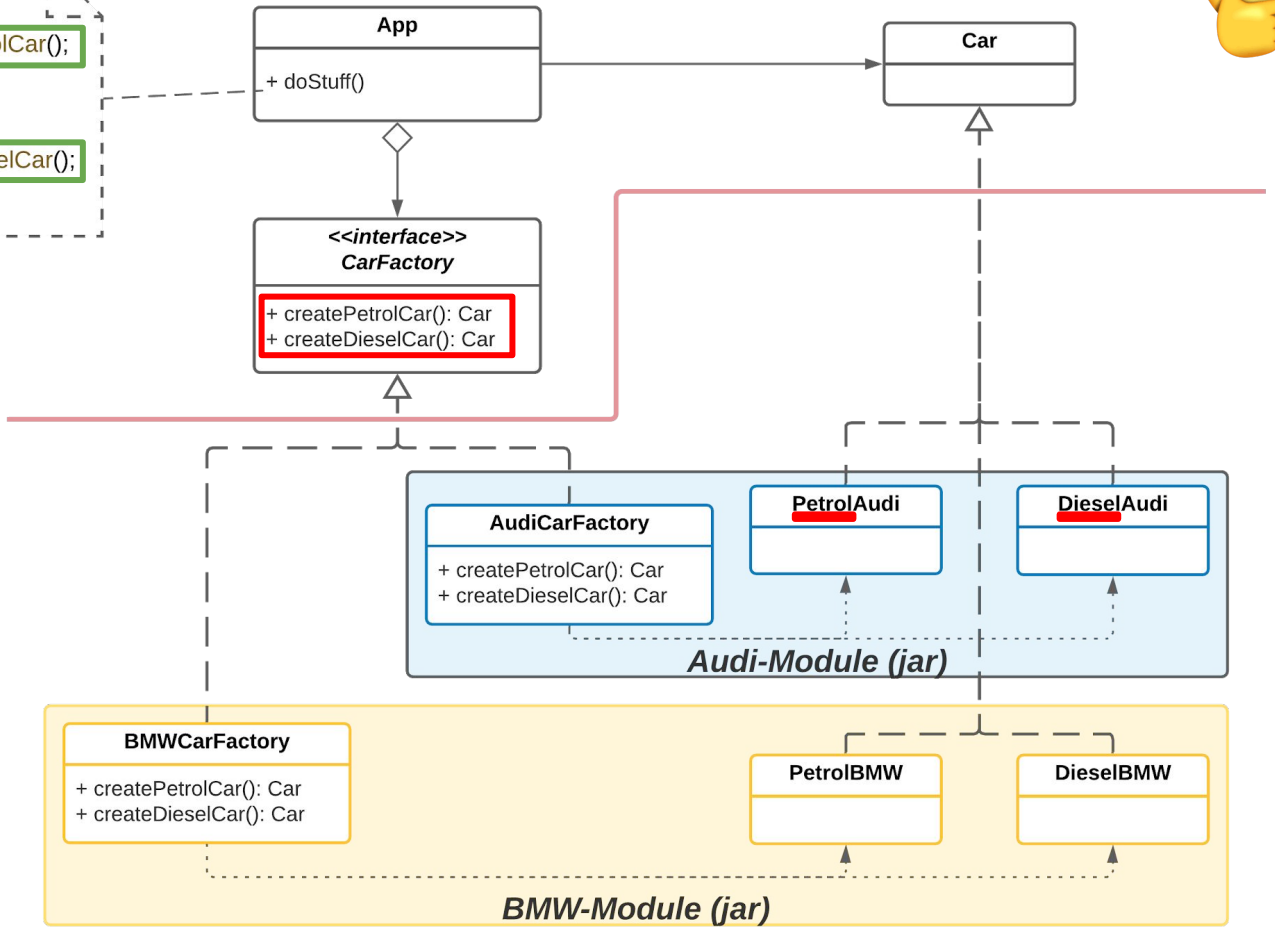
... // do something
```



Problem?

```
... // do something
Car petrolCar = factory.createPetrolCar();
... // do something
Car dieselCar = factory.createDieselCar();
... // do something
```





2 is the number of Car Types

How do you add an Electric Car?



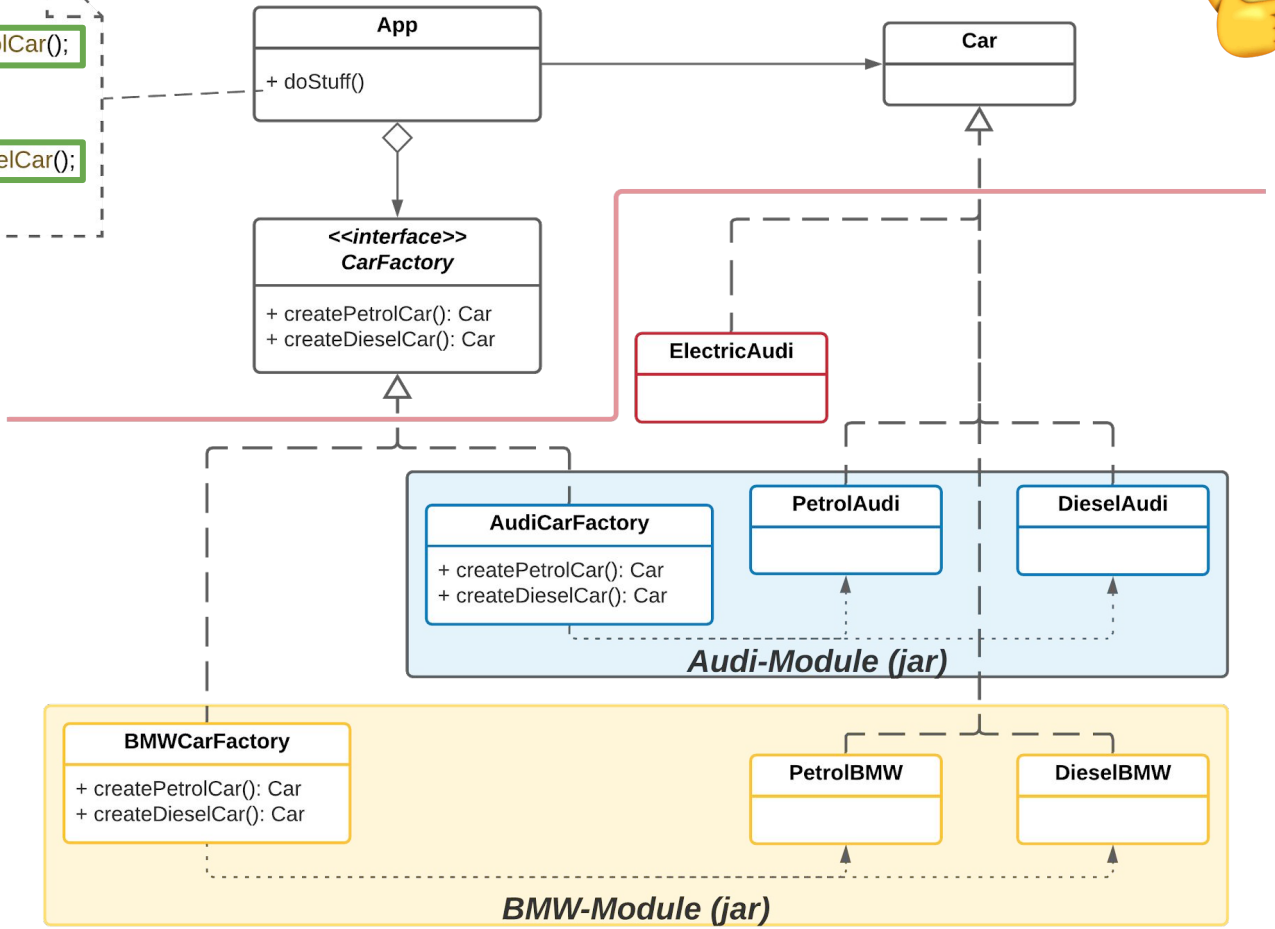
```
... // do something

Car petrolCar = factory.createPetrolCar();

... // do something

Car dieselCar = factory.createDieselCar();

... // do something
```



Use only 1 method...

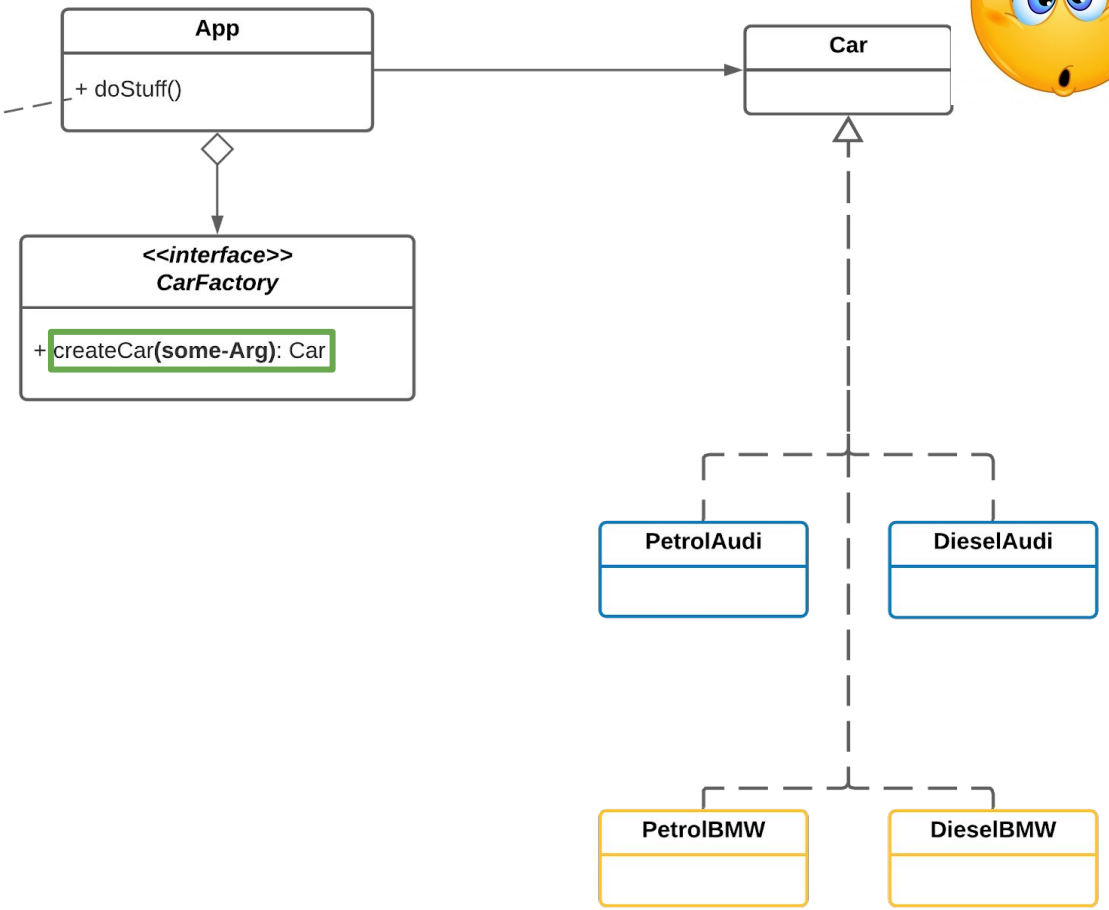
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```





Use only 1 method...



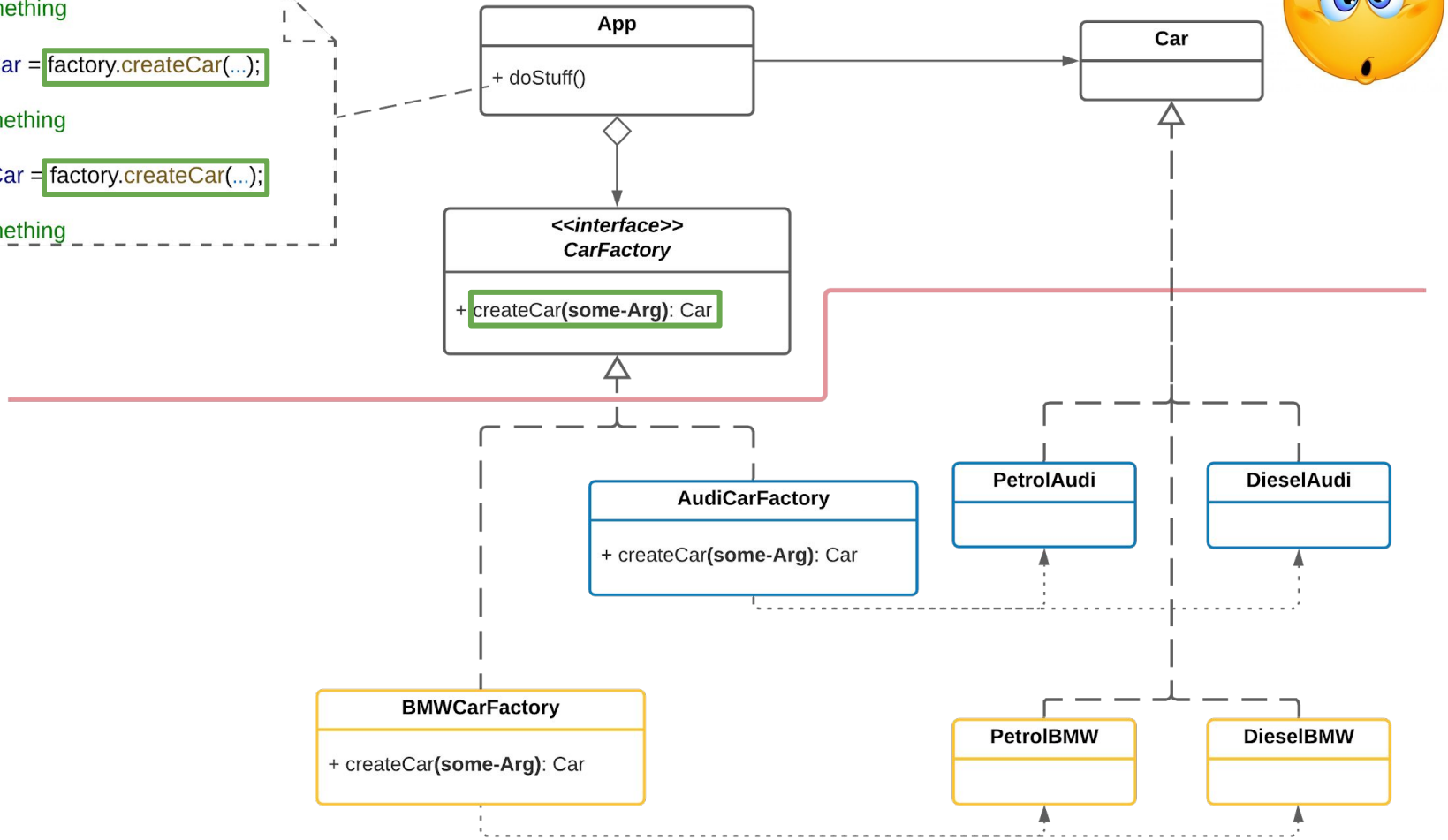
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```



What parameter should we pass in?



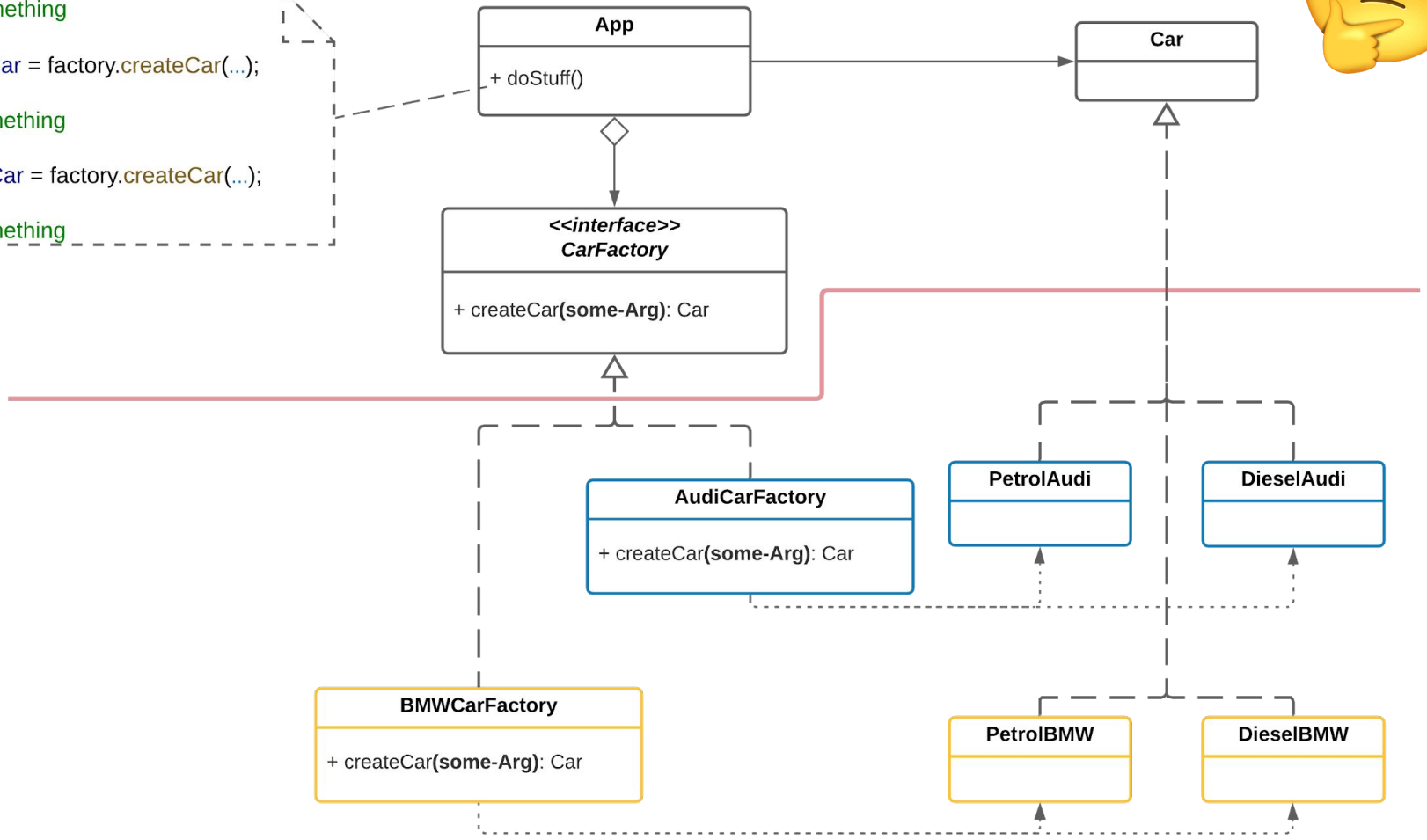
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```



Enum?

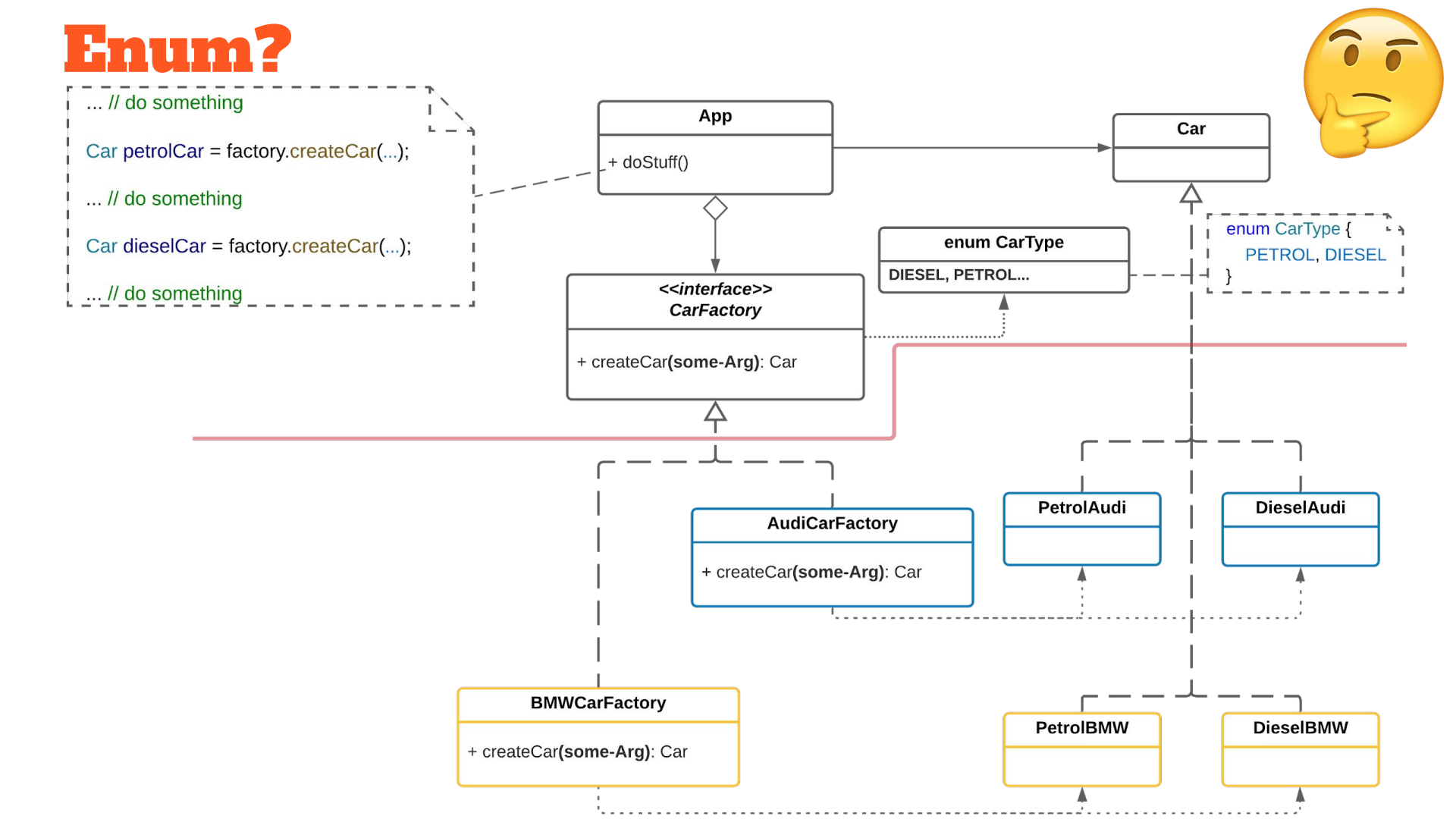
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```



Enum...



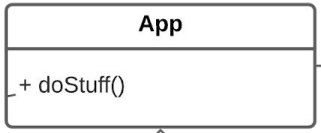
```
... // do something

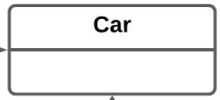
Car petrolCar = factory.createCar(PETROL);

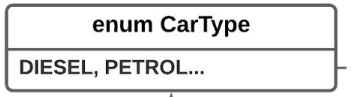
... // do something

Car dieselCar = factory.createCar(DIESEL);

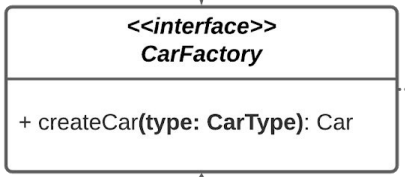
... // do something
```



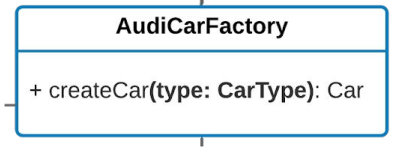


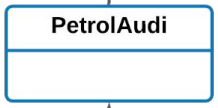


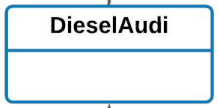

```
enum CarType {
    PETROL, DIESEL
}
```



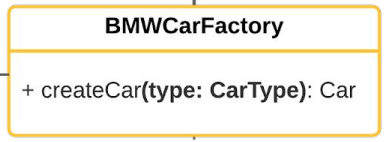

```
switch(type) {
    case PETROL: return new PetrolAudi();
    case DIESEL: return new DieselAudi();
    default: throw new UnknownCarException();
}
```

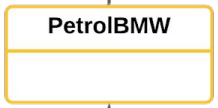


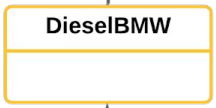





```
switch(type) {
    case PETROL: return new PetrolBMW();
    case DIESEL: return new DieselBMW();
    default: throw new UnknownCarException();
}
```







How do we add Electric Car?



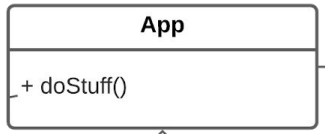
```
... // do something

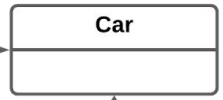
Car petrolCar = factory.createCar(PETROL);

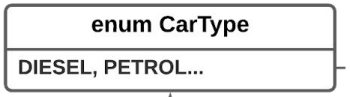
... // do something

Car dieselCar = factory.createCar(DIESEL);

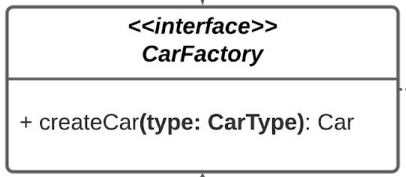
... // do something
```

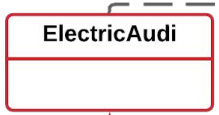




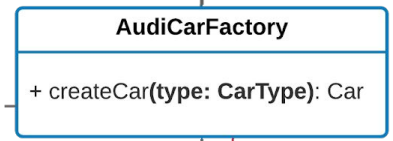


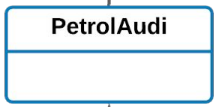

```
enum CarType {
    PETROL, DIESEL
}
```

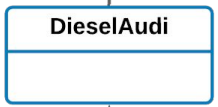




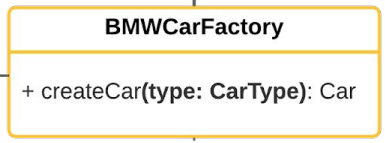

```
switch(type) {
    case PETROL: return new PetrolAudi();
    case DIESEL: return new DieselAudi();
    default: throw new UnknownCarException();
}
```

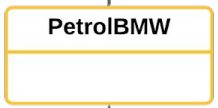


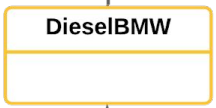





```
switch(type) {
    case PETROL: return new PetrolBMW();
    case DIESEL: return new DieselBMW();
    default: throw new UnknownCarException();
}
```







We need to modify the enum...



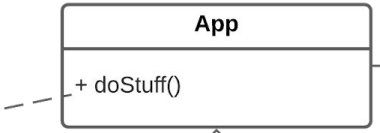
```
... // do something

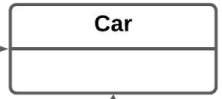
Car petrolCar = factory.createCar(PETROL);

... // do something

Car dieselCar = factory.createCar(DIESEL);

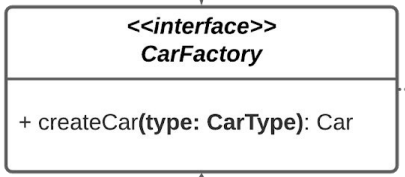
... // do something
```

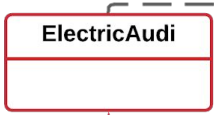




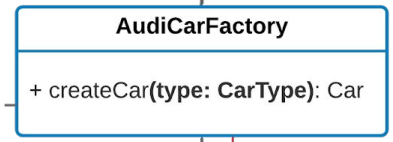


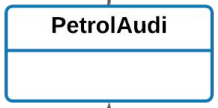

```
enum CarType {
    PETROL, DIESEL,
    ELECTRIC
}
```

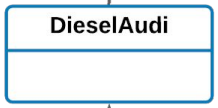




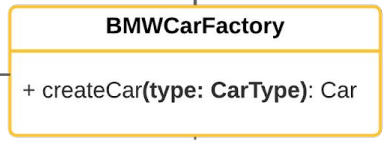

```
switch(type) {
    case PETROL: return new PetrolAudi();
    case DIESEL: return new DieselAudi();
    default: throw new UnknownCarException();
}
```

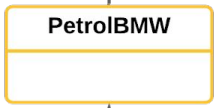


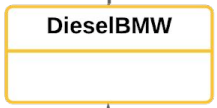





```
switch(type) {
    case PETROL: return new PetrolBMW();
    case DIESEL: return new DieselBMW();
    default: throw new UnknownCarException();
}
```







Enum is “static”...does not work....



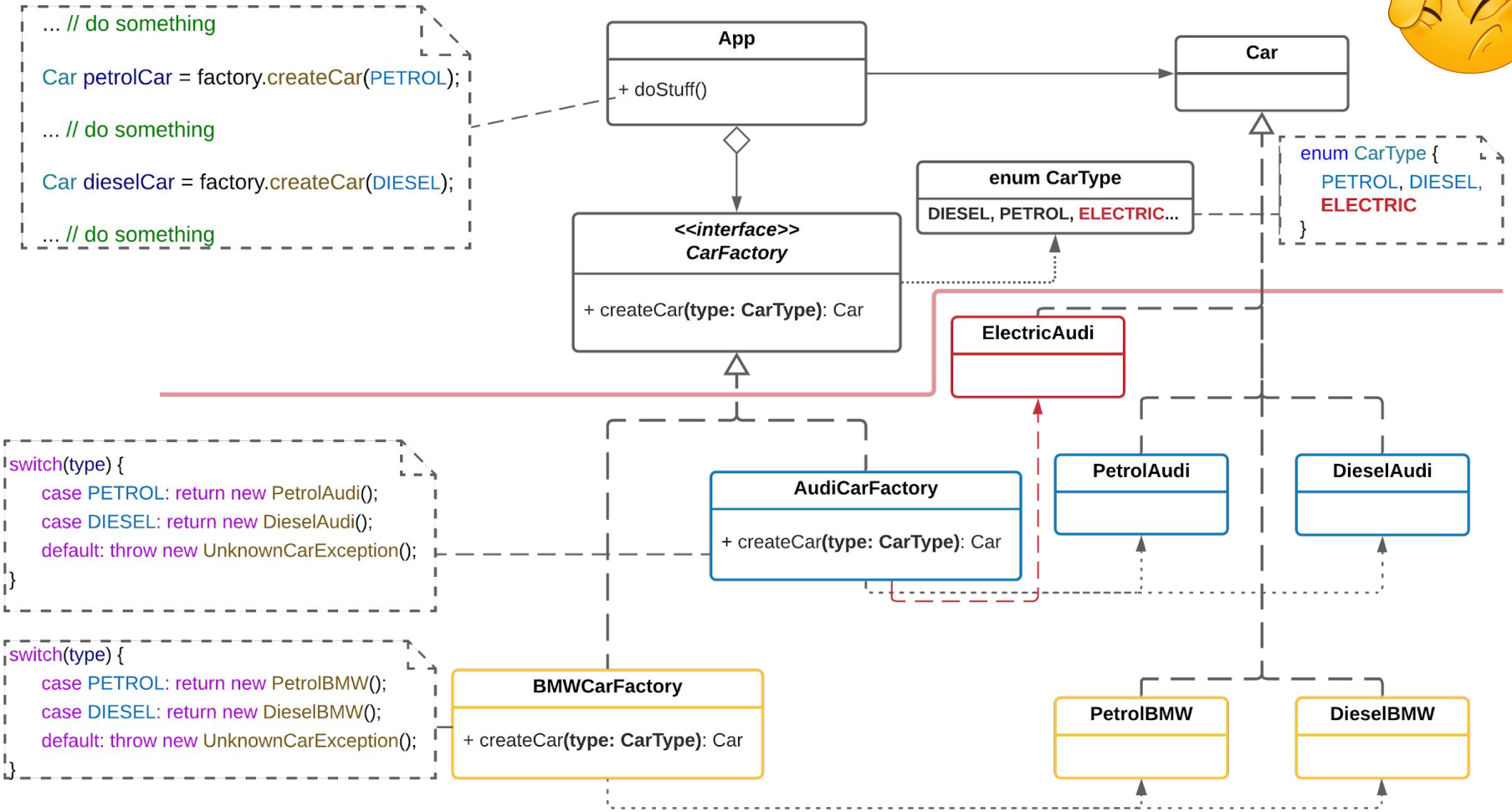
```
... // do something

Car petrolCar = factory.createCar(PETROL);

... // do something

Car dieselCar = factory.createCar(DIESEL);

... // do something
```



What parameter should we pass in?



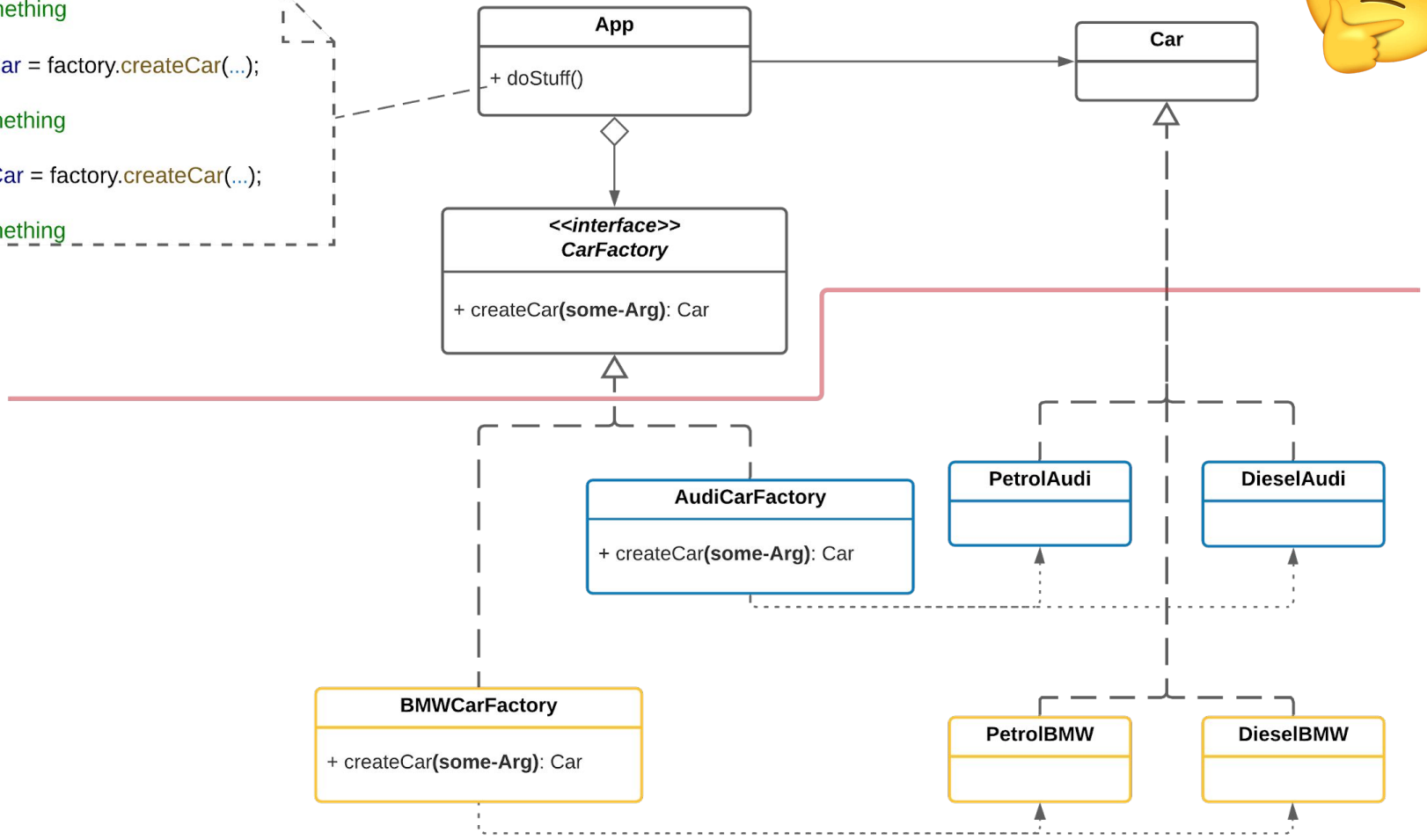
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```



String?...

```
... // do something

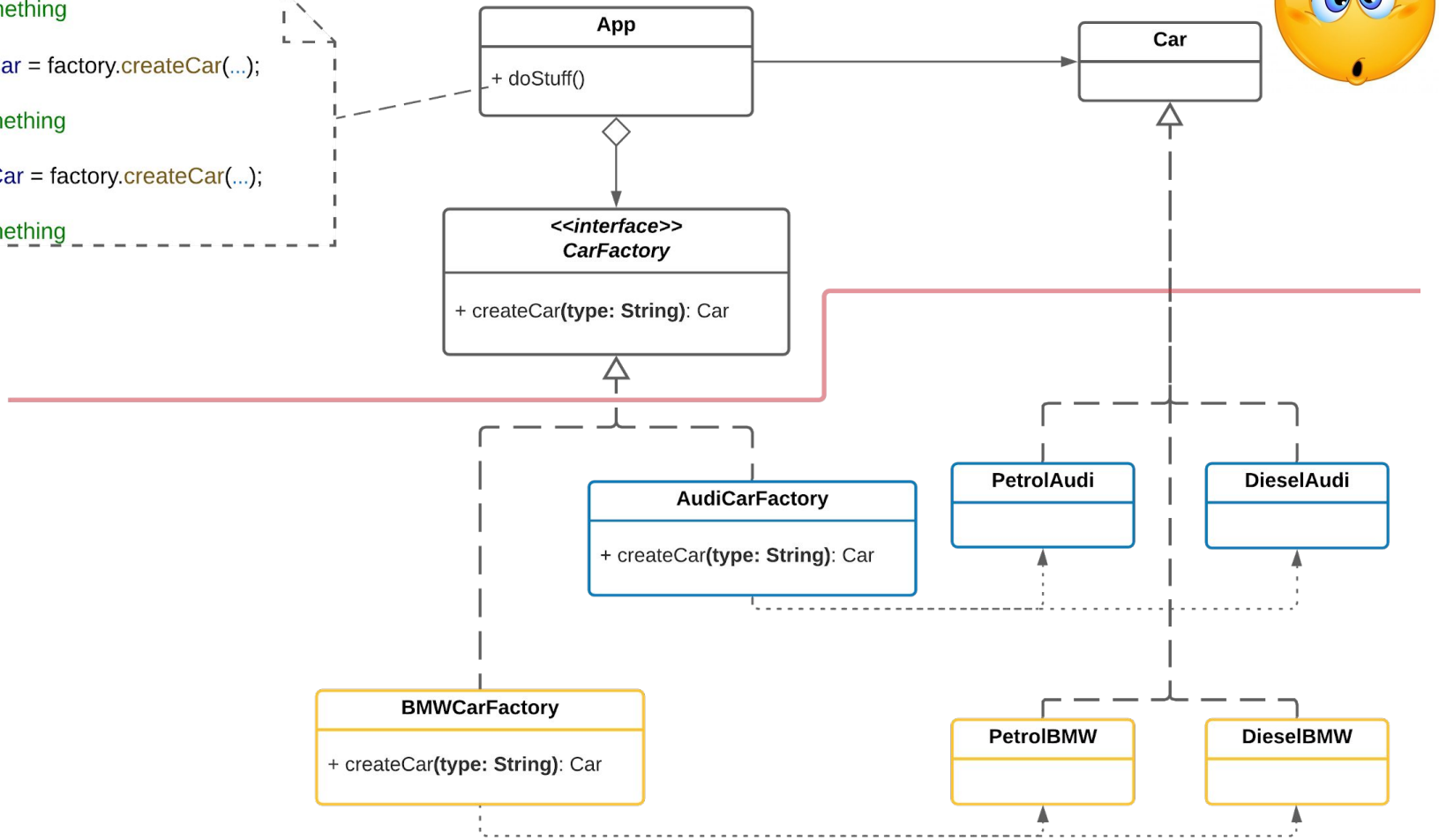
Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```





String?...It's not type safe...



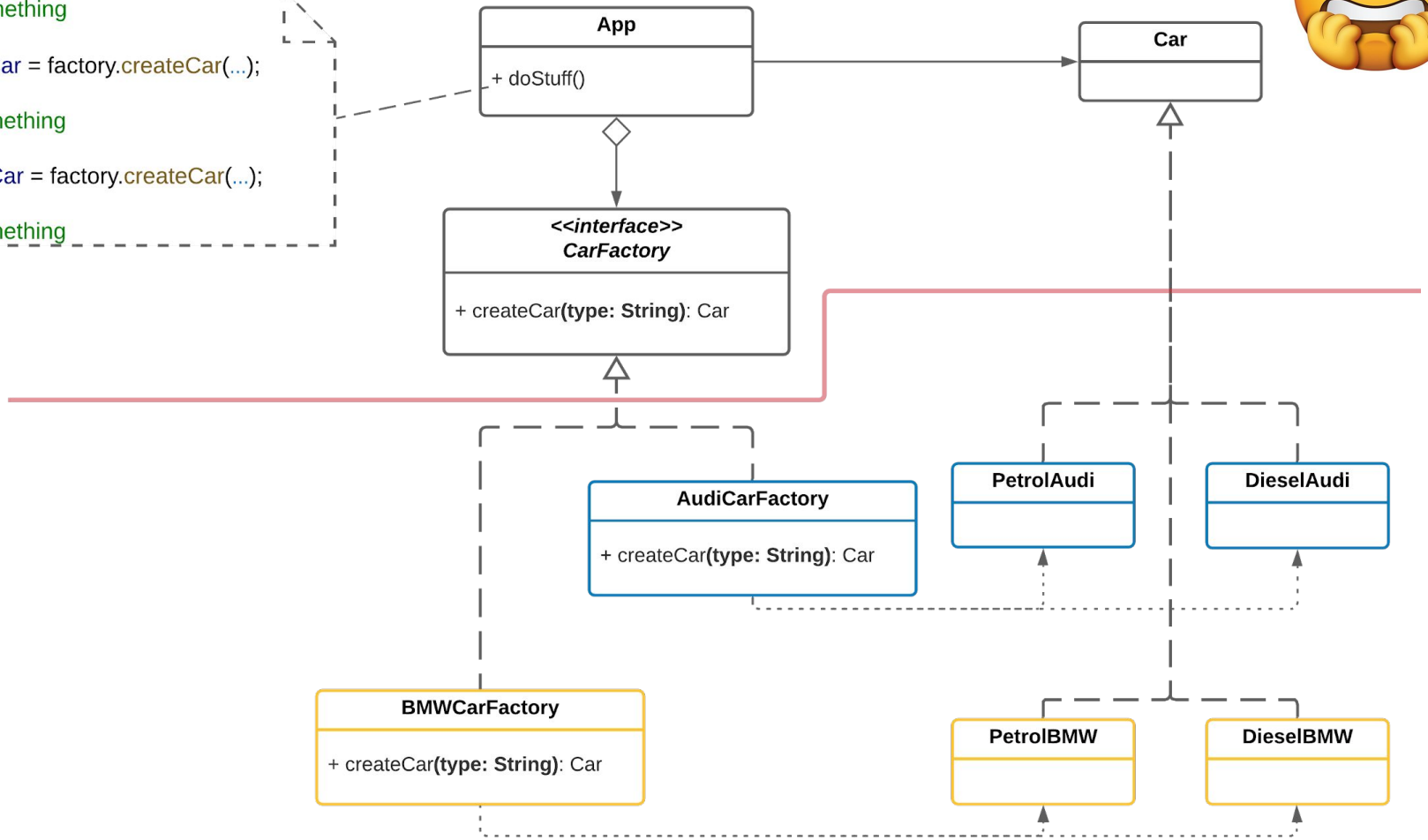
```
... // do something

Car petrolCar = factory.createCar(...);

... // do something

Car dieselCar = factory.createCar(...);

... // do something
```



We throw an exception...



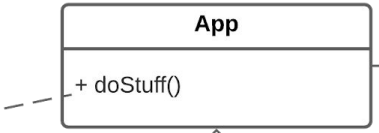
```
... // do something

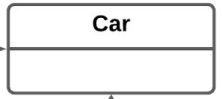
Car petrolCar = factory.createCar("PETROL");

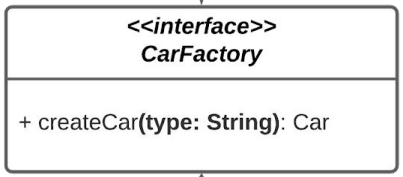
... // do something

Car dieselCar = factory.createCar("DIESEL");

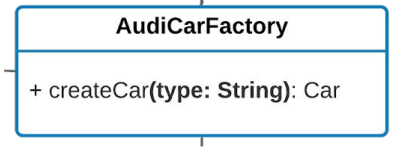
... // do something
```

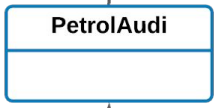


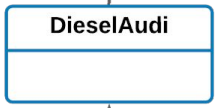




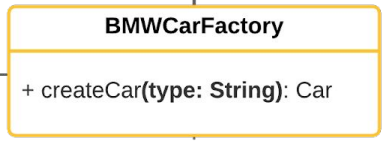

```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    default: throw new UnknownCarException();
}
```

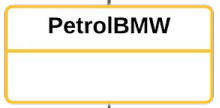


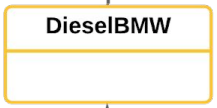





```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```







What types does a factory support?



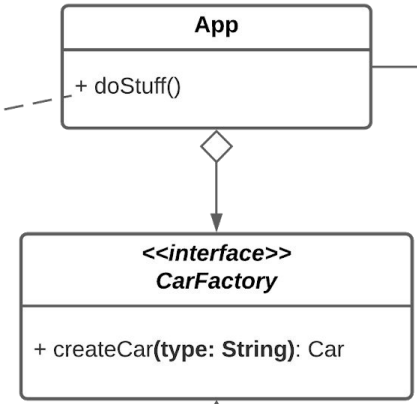
```
... // do something

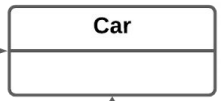
Car petrolCar = factory.createCar("PETROL");

... // do something

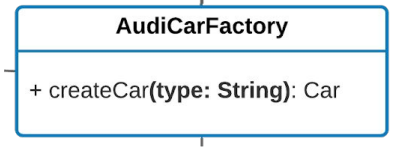
Car dieselCar = factory.createCar("DIESEL");

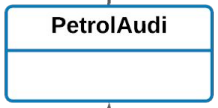
... // do something
```

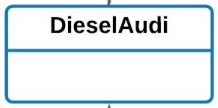




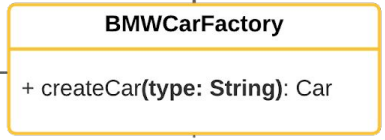

```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    default: throw new UnknownCarException();
}
```

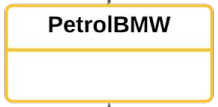


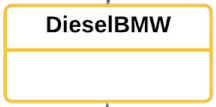





```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```







We let them tell us...



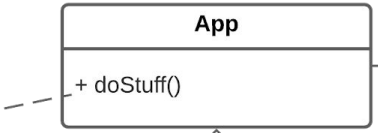
```
... // do something

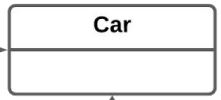
Car petrolCar = factory.createCar("PETROL");

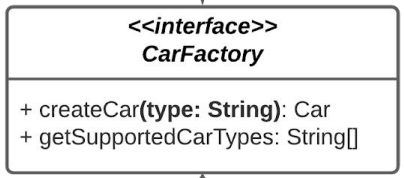
... // do something

Car dieselCar = factory.createCar("DIESEL");

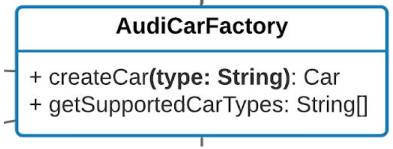
... // do something
```

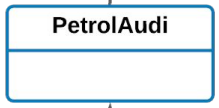


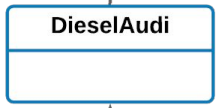





```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    default: throw new UnknownCarException();
}
```



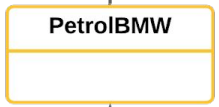


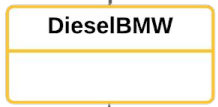



```
return new String[] {"PETROL", "DIESEL"};
```

```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```







But now how do we add Electric Car?



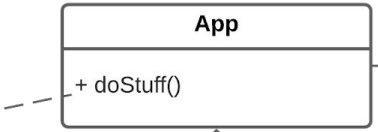
```
... // do something

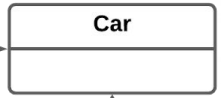
Car petrolCar = factory.createCar("PETROL");

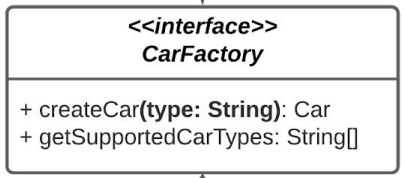
... // do something

Car dieselCar = factory.createCar("DIESEL");

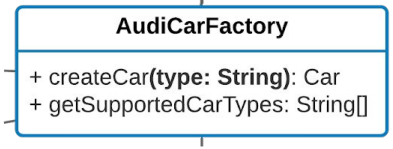
... // do something
```

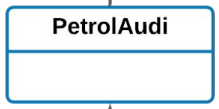


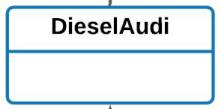





```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    default: throw new UnknownCarException();
}
```

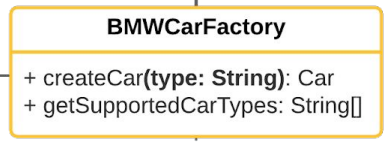


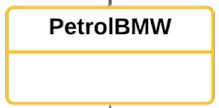


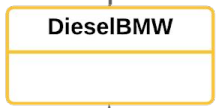



```
return new String[] {"PETROL", "DIESEL"};
```

```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```







We just add it and modify the factory



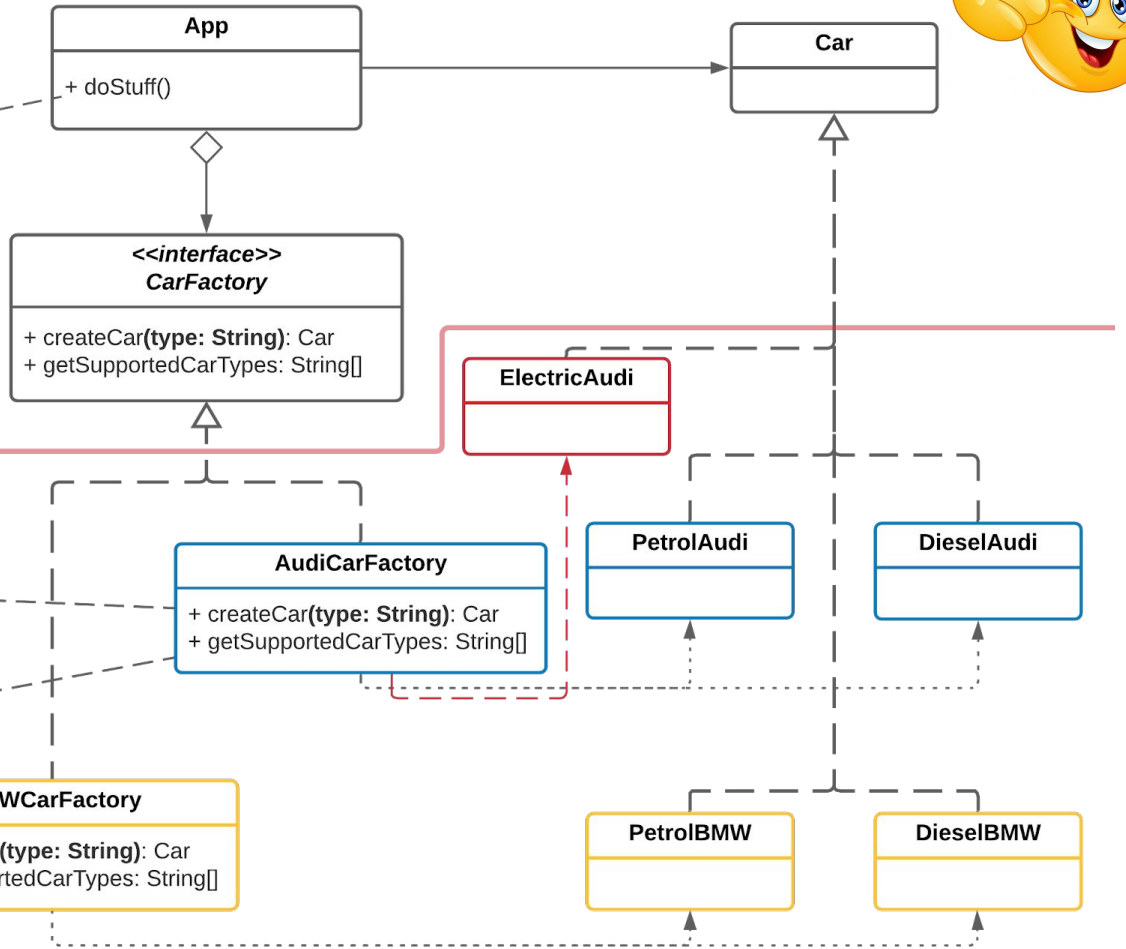
```
... // do something

Car petrolCar = factory.createCar("PETROL");

... // do something

Car dieselCar = factory.createCar("DIESEL");

... // do something
```




```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    case "ELECTRIC": return new ElectricAudi();
    default: throw new UnknownCarException();
}
```

```
return new String[] {"PETROL", "DIESEL", "ELECTRIC"};
```

```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```

What if we want to support all cars?



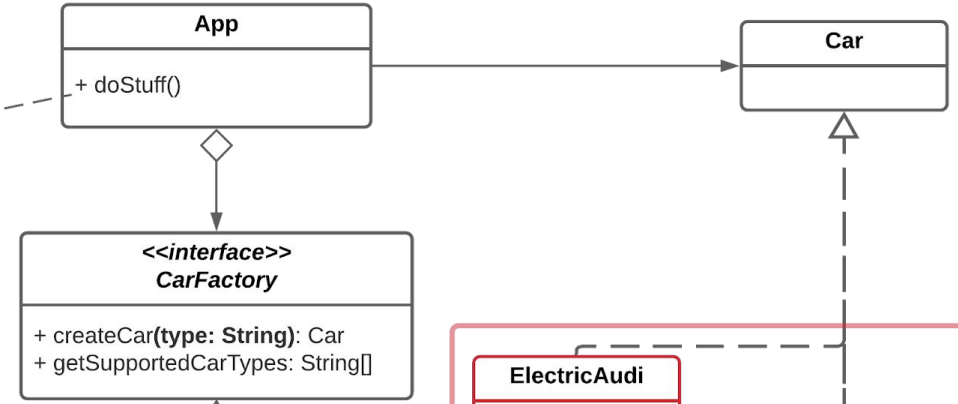
```
... // do something

Car petrolCar = factory.createCar("PETROL");

... // do something

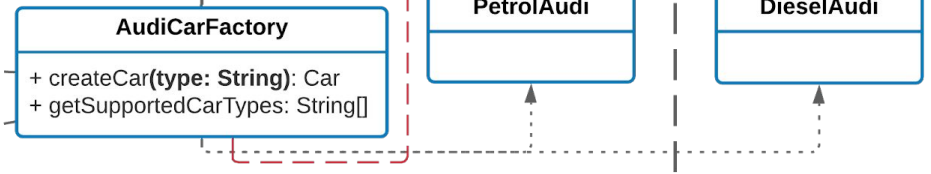
Car dieselCar = factory.createCar("DIESEL");

... // do something
```




```
switch(type) {
    case "PETROL": return new PetrolAudi();
    case "DIESEL": return new DieselAudi();
    case "ELECTRIC": return new ElectricAudi();
    default: throw new UnknownCarException();
}
```

```
return new String[] {"PETROL", "DIESEL", "ELECTRIC"};
```




```
switch(type) {
    case "PETROL": return new PetrolBMW();
    case "DIESEL": return new DieselBMW();
    default: throw new UnknownCarException();
}
```



We simply add a new factory!



```
... // do something

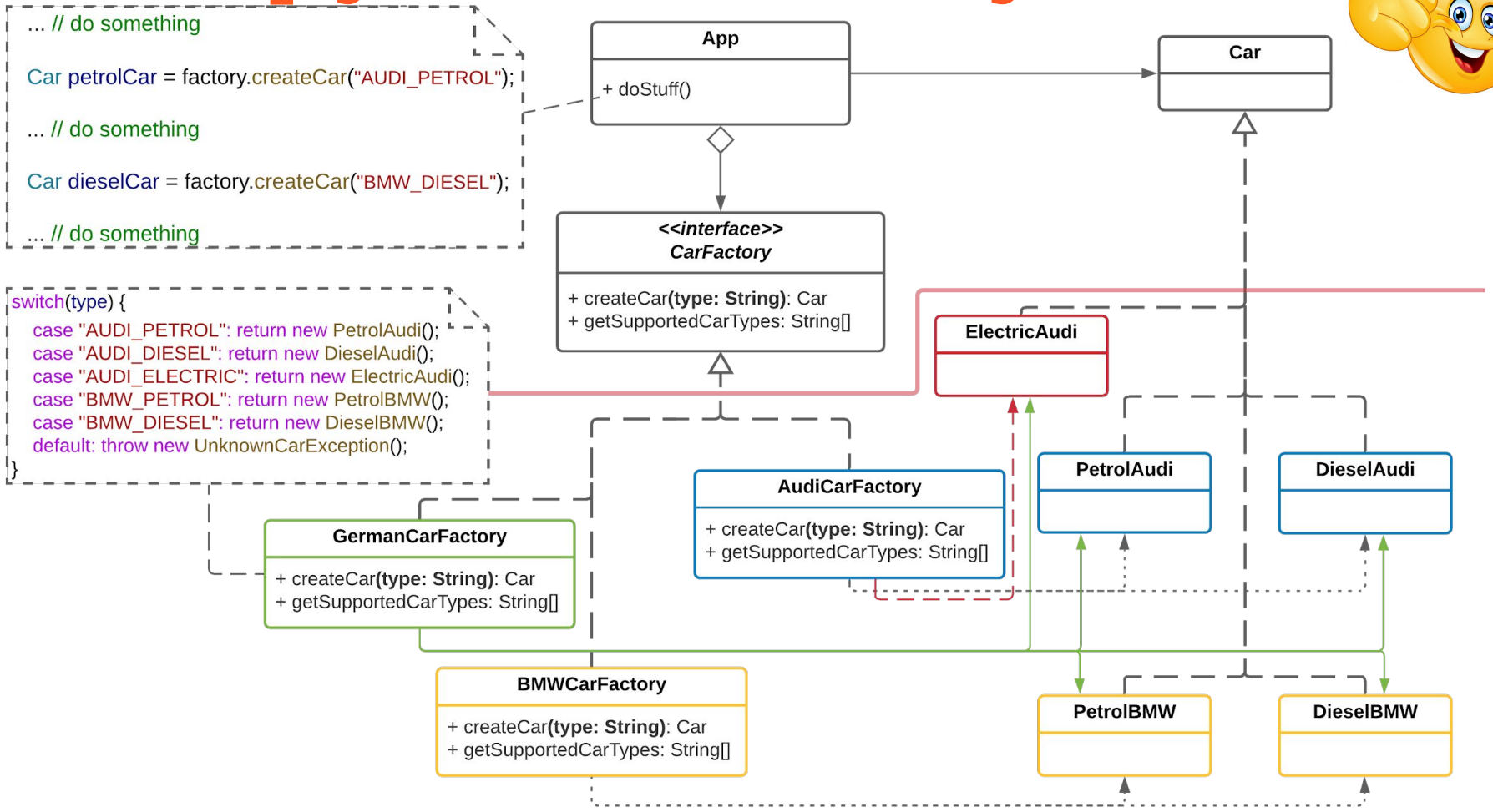
Car petrolCar = factory.createCar("AUDI_PETROL");

... // do something

Car dieselCar = factory.createCar("BMW_DIESEL");

... // do something
```

```
switch(type) {
  case "AUDI_PETROL": return new PetrolAudi();
  case "AUDI_DIESEL": return new DieselAudi();
  case "AUDI_ELECTRIC": return new ElectricAudi();
  case "BMW_PETROL": return new PetrolBMW();
  case "BMW_DIESEL": return new DieselBMW();
  default: throw new UnknownCarException();
}
```



And...we are done with this



```
... // do something

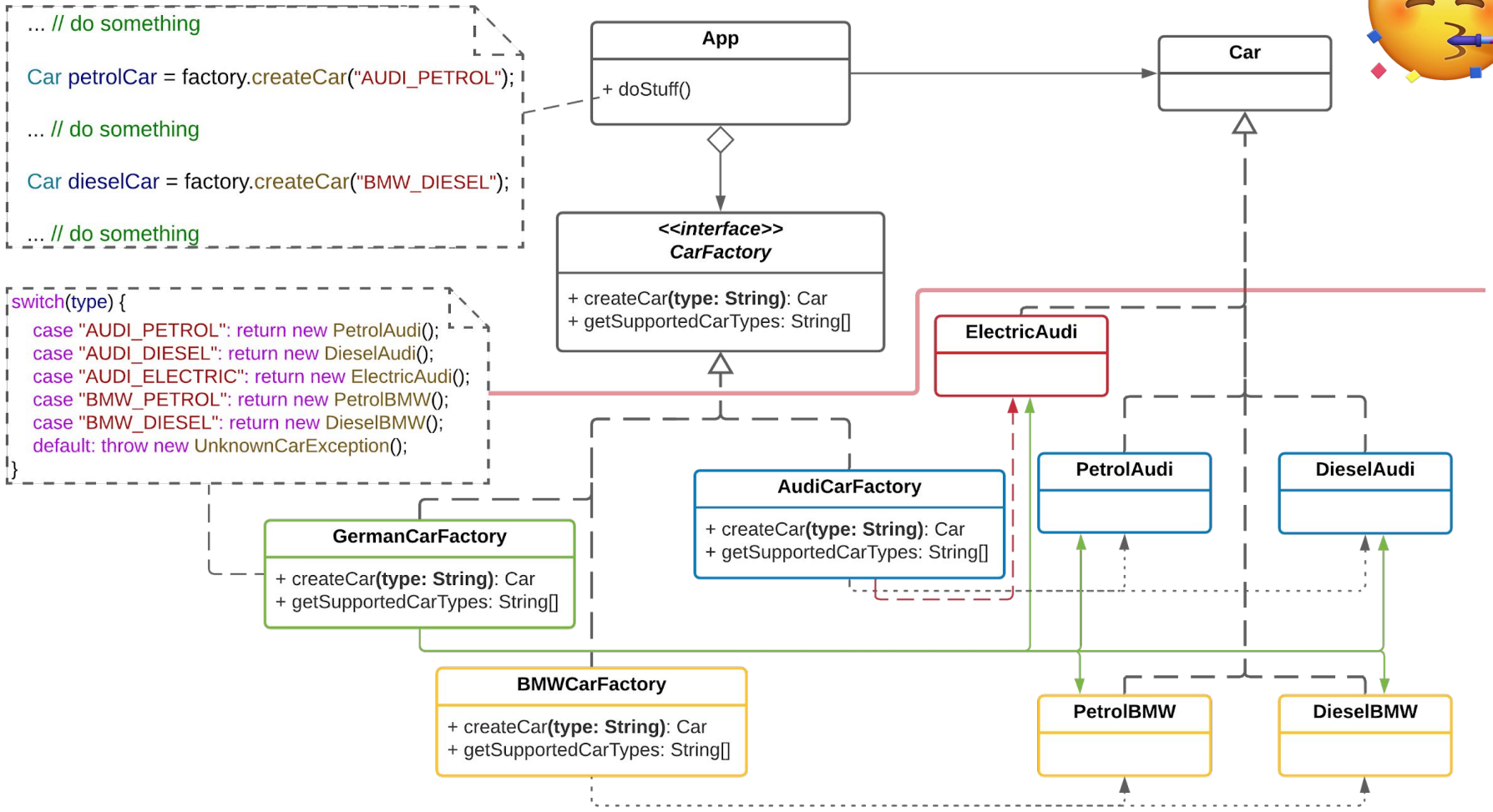
Car petrolCar = factory.createCar("AUDI_PETROL");

... // do something

Car dieselCar = factory.createCar("BMW_DIESEL");

... // do something
```

```
switch(type) {
  case "AUDI_PETROL": return new PetrolAudi();
  case "AUDI_DIESEL": return new DieselAudi();
  case "AUDI_ELECTRIC": return new ElectricAudi();
  case "BMW_PETROL": return new PetrolBMW();
  case "BMW_DIESEL": return new DieselBMW();
  default: throw new UnknownCarException();
}
```



Parametrized Abstract Factory



```
... // do something

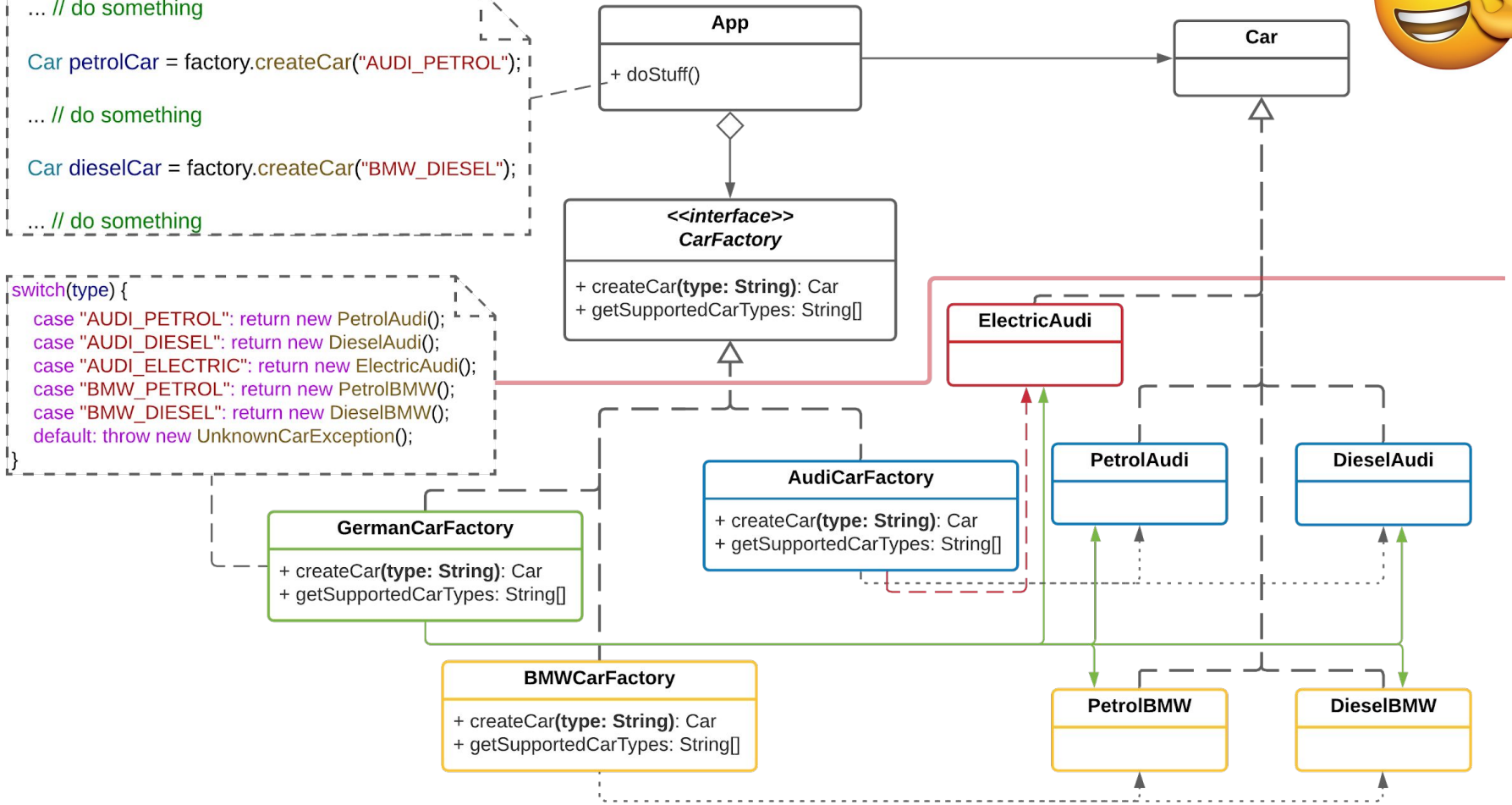
Car petrolCar = factory.createCar("AUDI_PETROL");

... // do something

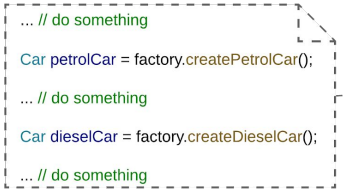
Car dieselCar = factory.createCar("BMW_DIESEL");

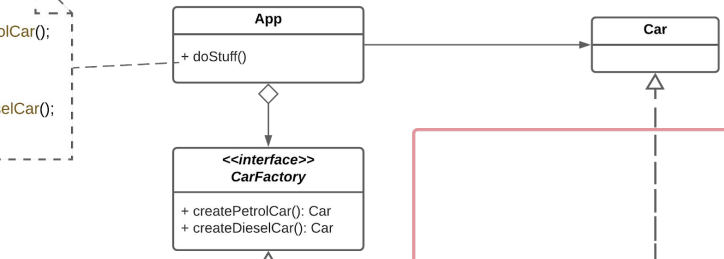
... // do something
```

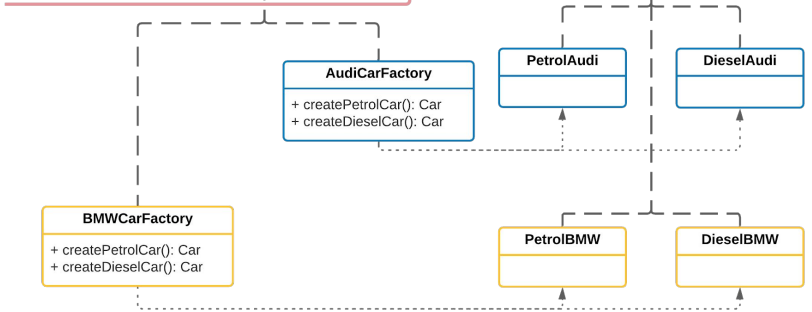
```
switch(type) {
  case "AUDI_PETROL": return new PetrolAudi();
  case "AUDI_DIESEL": return new DieselAudi();
  case "AUDI_ELECTRIC": return new ElectricAudi();
  case "BMW_PETROL": return new PetrolBMW();
  case "BMW_DIESEL": return new DieselBMW();
  default: throw new UnknownCarException();
}
```



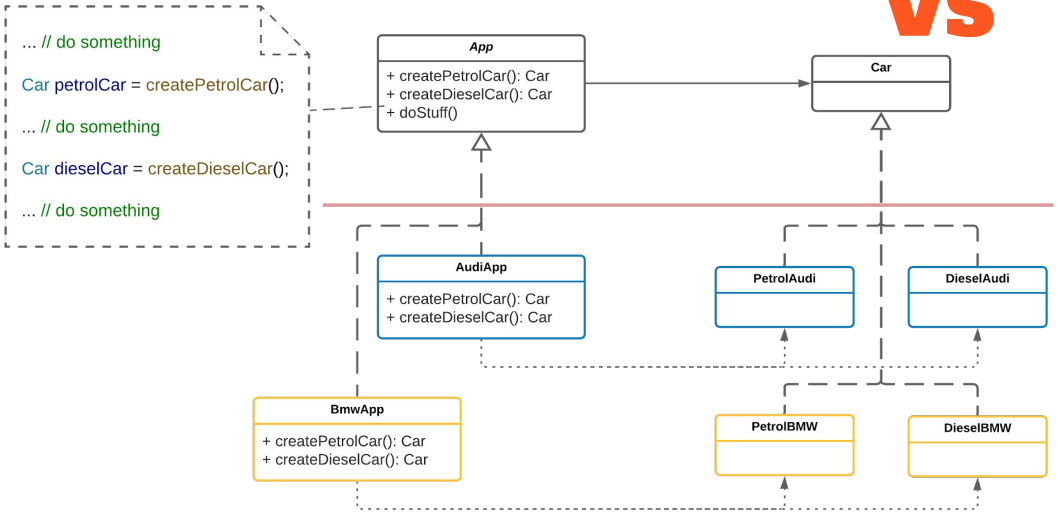
Factory Method







VS



Abstract Factory

Singleton

```
class Singleton {

    private static Singleton _instance = null;

    private Singleton() {}

    public static Singleton getInstance() {
        if (_instance == null) {
            _instance = new Singleton();
        }

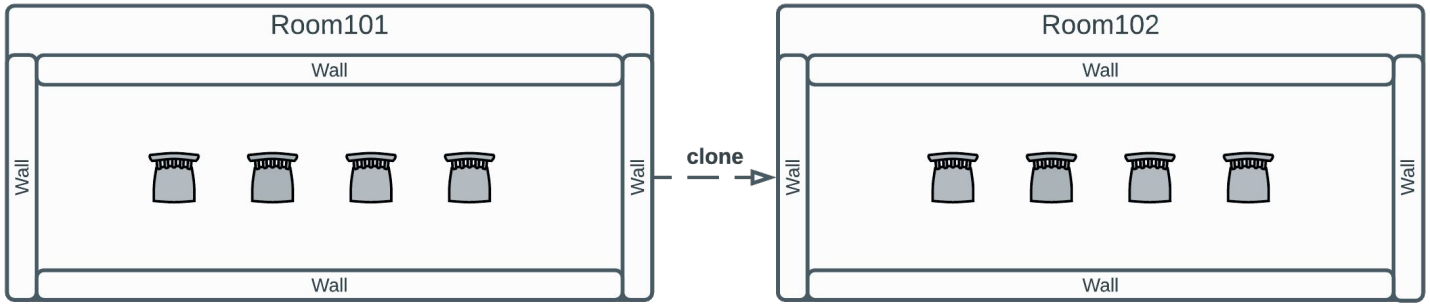
        return _instance;
    }
}
```

Prototype

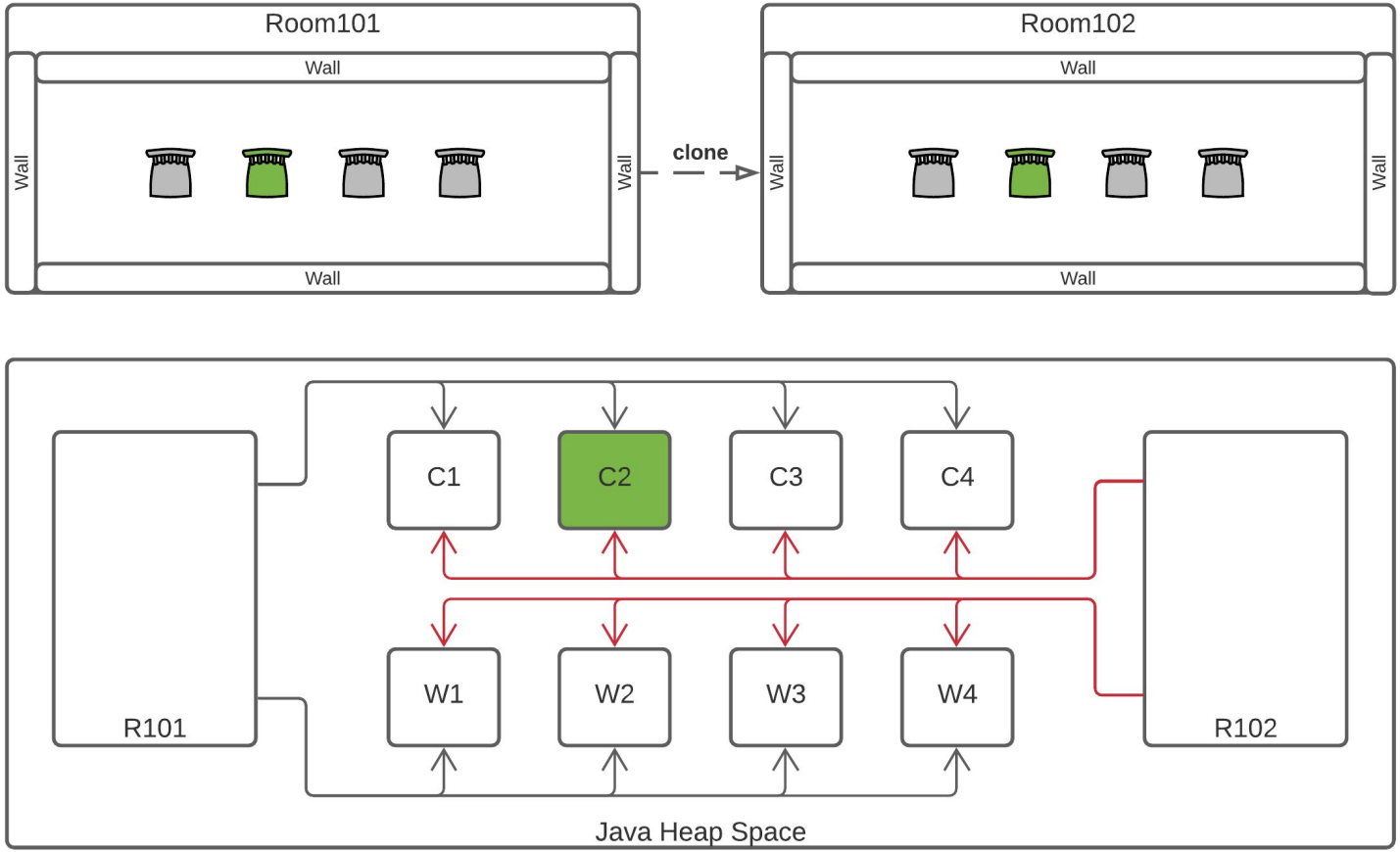



```
// do stuff ...
Shape newTriangle = shapeSeeds["triangle"].clone();
// do stuff ...
Shape newCircle = shapeSeeds["circle"].clone();
// do stuff ...
Shape newSquare = shapeSeeds["square"].clone();
// do stuff ...
```

Cloning Example



Shallow Cloning Example



Deep Cloning Example



Builder

```java
@AllArgsConstructor
class Student {
  private String firstName;
  private String lastName;
  private Integer age;
  private Student bestFriend;

  public static StudentBuilder builder() {
      return new StudentBuilder();
  }

  public static class StudentBuilder {
      private String firstName;
      private String lastName;
      private Integer age;
      private Student bestFriend;

      StudentBuilder() { }

      public StudentBuilder firstName(String firstName) { this.firstName = firstName; return this; }

      public StudentBuilder lastName(String lastName) { this.lastName = lastName; return this; }

      public StudentBuilder age(Integer age) { this.age = age; return this; }

      public StudentBuilder bestFriend(Student bestFriend) { this.bestFriend = bestFriend; return this; }

      public Student build() { return new Student(firstName, lastName, age, bestFriend); }
  }
}
```

Creational Patterns Wrap up

Summary - Creational Patterns

- They solve the problem of creation, while preserving flexibility (DIP, OCP...)
- Prototype uses cloning to create objects
- In Java, for an object to be able to call the inherited method *clone()* (from the Object class), the class needs to implement *Cloneable*
- Shallow vs. Deep cloning
- Singleton can be used to create a fixed number of objects of a given type
- Builder is used to simplify the creation of complex objects

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**



Structural

- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight

Behavioral

- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State

Resources

- https://sourcemaking.com/design_patterns
- https://refactoring.guru/design-patterns
- ***Design Patterns: Elements of Reusable Object-Oriented Software***, written by *Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides*